```
import logging
import math
from typing import Any, Dict, List, Optional, Tuple


LOGGER = logging.getLogger("TRADER_AI_PRO.STRATEGY")
logger = LOGGER  # Alias de compatibilite utilise par les fonctions du module.


SYMBOL_NAMES = {
    "1HZ10V": "Volatility 10 (1s) Index",
    "1HZ15V": "Volatility 15 (1s) Index",
    "1HZ25V": "Volatility 25 (1s) Index",
    "1HZ30V": "Volatility 30 (1s) Index",
    "1HZ50V": "Volatility 50 (1s) Index",
    "1HZ75V": "Volatility 75 (1s) Index",
    "1HZ90V": "Volatility 90 (1s) Index",
    "1HZ100V": "Volatility 100 (1s) Index",
    "R_10": "Volatility 10 Index",
    "R_25": "Volatility 25 Index",
    "R_50": "Volatility 50 Index",
    "R_75": "Volatility 75 Index",
    "R_100": "Volatility 100 Index",
}


TIMEFRAME_REQUIREMENTS = {
    "D1": 200,
    "H4": 200,
    "H2": 200,
    "H1": 200,
}


MIN_TRENDLINE_TOUCHES = 3
ATR_PERIOD = 14
MIN_RISK_REWARD = 1.0
ATR_STOP_BUFFER = 0.15
ZONE_ATR_TOLERANCE = 0.25
MIN_CANDLE_COUNT = 200
MAX_PRICE_DISTANCE_ATR = 1.5


class StrategyError(Exception):
    pass


def get_symbol_name(symbol: str) -> str:
    """Renvoie le nom lisible d'un indice Deriv."""
    symbol = str(symbol).strip()
    return SYMBOL_NAMES.get(symbol, symbol)


def _finite_number(
    value: Any,
```

```
    field_name: str = "valeur",
) -> float:
    """Convertit une valeur en nombre fini."""
    try:
        number = float(value)
    except (TypeError, ValueError) as exc:
        raise StrategyError(
            f"Valeur numérique invalide pour {field_name}."
        ) from exc

    if not math.isfinite(number):
        raise StrategyError(
            f"Valeur non finie pour {field_name}."
        )

    return number


def _validate_candle(
    candle: Dict[str, Any],
) -> Dict[str, float]:
    """Valide et normalise une bougie OHLC."""
    if not isinstance(candle, dict):
        raise StrategyError(
            "Une bougie doit être un dictionnaire."
        )

    try:
        epoch = int(candle["epoch"])
        open_price = _finite_number(
            candle["open"],
            "open",
        )
        high_price = _finite_number(
            candle["high"],
            "high",
        )
        low_price = _finite_number(
            candle["low"],
            "low",
        )
        close_price = _finite_number(
            candle["close"],
            "close",
        )
    except KeyError as exc:
        raise StrategyError(
            f"Champ de bougie manquant : {exc}"
        ) from exc

    if epoch <= 0:
        raise StrategyError(
            "Horodatage de bougie invalide."
        )

    if low_price > high_price:
        raise StrategyError(
            "Le minimum dépasse le maximum de la bougie."
```

```
        )

    if not (
        low_price
        <= min(open_price, close_price)
        <= max(open_price, close_price)
        <= high_price
    ):
        raise StrategyError(
            "Les prix OHLC de la bougie sont incohérents."
        )

    return {
        "epoch": epoch,
        "open": open_price,
        "high": high_price,
        "low": low_price,
        "close": close_price,
    }


def validate_candle_series(
    candles: List[Dict[str, Any]],
    minimum_count: int = MIN_CANDLE_COUNT,
) -> List[Dict[str, float]]:
    """Vérifie une série de bougies ordonnée et exploitable."""
    if not isinstance(candles, list):
        raise StrategyError(
            "La série de bougies doit être une liste."
        )

    if len(candles) < minimum_count:
        raise StrategyError(
            f"Historique insuffisant : {len(candles)} bougies "
            f"reçues, {minimum_count} nécessaires."
        )

    normalized = [
        _validate_candle(candle)
        for candle in candles
    ]

    normalized.sort(
        key=lambda candle: candle["epoch"]
    )

    previous_epoch = 0

    for candle in normalized:
        if candle["epoch"] <= previous_epoch:
            raise StrategyError(
                "Les horodatages des bougies doivent être uniques."
            )

        previous_epoch = int(candle["epoch"])

    return normalized
```

```
def calculate_true_range(
    current: Dict[str, float],
    previous_close: Optional[float],
) -> float:
    """Calcule le True Range d'une bougie."""
    high_price = float(current["high"])
    low_price = float(current["low"])

    if previous_close is None:
        return high_price - low_price

    return max(
        high_price - low_price,
        abs(high_price - previous_close),
        abs(low_price - previous_close),
    )


def calculate_atr(
    candles: List[Dict[str, Any]],
    period: int = ATR_PERIOD,
) -> float:
    """Calcule l'ATR à partir des dernières bougies."""
    if period < 1:
        raise StrategyError(
            "La période ATR doit être positive."
        )

    normalized = validate_candle_series(
        candles,
        minimum_count=period + 1,
    )

    true_ranges = []

    for index, candle in enumerate(normalized):
        previous_close = None

        if index > 0:
            previous_close = normalized[index - 1]["close"]

        true_ranges.append(
            calculate_true_range(
                candle,
                previous_close,
            )
        )

    recent_ranges = true_ranges[-period:]

    if not recent_ranges:
        raise StrategyError(
            "Impossible de calculer l'ATR."
        )

    atr = sum(recent_ranges) / len(recent_ranges)
```

```
    if not math.isfinite(atr) or atr <= 0:
        raise StrategyError(
            "La valeur ATR calculée est invalide."
        )

    return atr


def candle_body_ratio(
    candle: Dict[str, Any],
) -> float:
    """Mesure la proportion du corps dans l'amplitude totale."""
    normalized = _validate_candle(candle)

    total_range = (
        normalized["high"] - normalized["low"]
    )

    if total_range <= 0:
        return 0.0

    body = abs(
        normalized["close"] - normalized["open"]
    )

    return body / total_range


def candle_direction(
    candle: Dict[str, Any],
) -> str:
    """Détermine la direction d'une bougie."""
    normalized = _validate_candle(candle)

    if normalized["close"] > normalized["open"]:
        return "BULLISH"

    if normalized["close"] < normalized["open"]:
        return "BEARISH"

    return "DOJI"


def is_bullish_candle(
    candle: Dict[str, Any],
) -> bool:
    return (
        candle_direction(candle) == "BULLISH"
    )


def is_bearish_candle(
    candle: Dict[str, Any],
) -> bool:
    return (
        candle_direction(candle) == "BEARISH"
    )
```

```
def candle_upper_wick(
    candle: Dict[str, Any],
) -> float:
    normalized = _validate_candle(candle)

    return (
        normalized["high"]
        - max(
            normalized["open"],
            normalized["close"],
        )
    )


def candle_lower_wick(
    candle: Dict[str, Any],
) -> float:
    normalized = _validate_candle(candle)

    return (
        min(
            normalized["open"],
            normalized["close"],
        )
        - normalized["low"]
    )


def is_doji(
    candle: Dict[str, Any],
    maximum_body_ratio: float = 0.10,
) -> bool:
    """Détecte un doji selon la proportion de son corps."""
    if not 0 <= maximum_body_ratio <= 1:
        raise StrategyError(
            "Le ratio maximal du doji doit être compris entre 0 et 1."
        )

    return (
        candle_body_ratio(candle)
        <= maximum_body_ratio
    )
def find_pivot_highs(
    candles: List[Dict[str, Any]],
    radius: int = 2,
) -> List[Dict[str, Any]]:
    """Identifie les sommets locaux servant aux résistances."""
    if radius < 1:
        raise StrategyError(
            "Le rayon des pivots doit être positif."
        )

    normalized = validate_candle_series(
        candles,
        minimum_count=(radius * 2) + 1,
    )
```

```
    pivots = []

    for index in range(
        radius,
        len(normalized) - radius,
    ):
        current = normalized[index]
        current_high = current["high"]

        left_highs = [
            candle["high"]
            for candle in normalized[
                index - radius:index
            ]
        ]

        right_highs = [
            candle["high"]
            for candle in normalized[
                index + 1:index + radius + 1
            ]
        ]

        if (
            current_high >= max(left_highs)
            and current_high >= max(right_highs)
            and (
                current_high > max(left_highs)
                or current_high > max(right_highs)
            )
        ):
            pivots.append(
                {
                    "index": index,
                    "epoch": current["epoch"],
                    "price": current_high,
                    "type": "HIGH",
                }
            )

    return pivots


def find_pivot_lows(
    candles: List[Dict[str, Any]],
    radius: int = 2,
) -> List[Dict[str, Any]]:
    """Identifie les creux locaux servant aux supports."""
    if radius < 1:
        raise StrategyError(
            "Le rayon des pivots doit être positif."
        )

    normalized = validate_candle_series(
        candles,
        minimum_count=(radius * 2) + 1,
    )
```

```
    pivots = []

    for index in range(
        radius,
        len(normalized) - radius,
    ):
        current = normalized[index]
        current_low = current["low"]

        left_lows = [
            candle["low"]
            for candle in normalized[
                index - radius:index
            ]
        ]

        right_lows = [
            candle["low"]
            for candle in normalized[
                index + 1:index + radius + 1
            ]
        ]

        if (
            current_low <= min(left_lows)
            and current_low <= min(right_lows)
            and (
                current_low < min(left_lows)
                or current_low < min(right_lows)
            )
        ):
            pivots.append(
                {
                    "index": index,
                    "epoch": current["epoch"],
                    "price": current_low,
                    "type": "LOW",
                }
            )

    return pivots


def cluster_price_levels(
    pivots: List[Dict[str, Any]],
    tolerance: float,
    level_type: str,
) -> List[Dict[str, Any]]:
    """Regroupe les pivots proches en zones techniques."""
    tolerance = _finite_number(
        tolerance,
        "tolerance",
    )

    if tolerance < 0:
        raise StrategyError(
            "La tolérance ne peut pas être négative."
        )
```

```
if level_type not in ("SUPPORT", "RESISTANCE"):
    raise StrategyError(
        "Type de niveau technique invalide."
    )

valid_pivots = []

for pivot in pivots:
    if not isinstance(pivot, dict):
        continue

    try:
        price = _finite_number(
            pivot["price"],
            "pivot.price",
        )

        index = int(pivot["index"])
        epoch = int(pivot["epoch"])

    except (KeyError, TypeError, ValueError, StrategyError):
        continue

    if price <= 0 or index < 0 or epoch <= 0:
        continue

    valid_pivots.append(
        {
            "price": price,
            "index": index,
            "epoch": epoch,
        }
    )

valid_pivots.sort(
    key=lambda pivot: pivot["price"]
)

clusters = []

for pivot in valid_pivots:
    matching_cluster = None

    for cluster in clusters:
        center = (
            sum(cluster["prices"])
            / len(cluster["prices"])
        )

        if abs(pivot["price"] - center) <= tolerance:
            matching_cluster = cluster
            break

    if matching_cluster is None:
        clusters.append(
            {
                "prices": [pivot["price"]],
```

```
                    "indices": [pivot["index"]],
                    "epochs": [pivot["epoch"]],
                }
            )
        else:
            matching_cluster["prices"].append(
                pivot["price"]
            )

            matching_cluster["indices"].append(
                pivot["index"]
            )

            matching_cluster["epochs"].append(
                pivot["epoch"]
            )

    levels = []

    for cluster in clusters:
        prices = cluster["prices"]

        lower = min(prices)
        upper = max(prices)
        center = sum(prices) / len(prices)

        levels.append(
            {
                "type": level_type,
                "lower": lower,
                "upper": upper,
                "price": center,
                "touches": len(prices),
                "indices": sorted(cluster["indices"]),
                "epochs": sorted(cluster["epochs"]),
                "strength": len(prices),
            }
        )

    levels.sort(
        key=lambda level: level["price"]
    )

    return levels


def find_supports(
    candles: List[Dict[str, Any]],
    atr: Optional[float] = None,
    radius: int = 2,
) -> List[Dict[str, Any]]:
    """Détecte les zones de support à partir des creux."""
    normalized = validate_candle_series(
        candles,
        minimum_count=(radius * 2) + 1,
    )

    if atr is None:
```

```
        atr = calculate_atr(normalized)

    atr = _finite_number(atr, "ATR")

    if atr <= 0:
        raise StrategyError(
            "L'ATR doit être positif pour détecter les supports."
        )

    pivots = find_pivot_lows(
        normalized,
        radius=radius,
    )

    return cluster_price_levels(
        pivots=pivots,
        tolerance=atr * ZONE_ATR_TOLERANCE,
        level_type="SUPPORT",
    )


def find_resistances(
    candles: List[Dict[str, Any]],
    atr: Optional[float] = None,
    radius: int = 2,
) -> List[Dict[str, Any]]:
    """Détecte les zones de résistance à partir des sommets."""
    normalized = validate_candle_series(
        candles,
        minimum_count=(radius * 2) + 1,
    )

    if atr is None:
        atr = calculate_atr(normalized)

    atr = _finite_number(atr, "ATR")

    if atr <= 0:
        raise StrategyError(
            "L'ATR doit être positif pour détecter les résistances."
        )

    pivots = find_pivot_highs(
        normalized,
        radius=radius,
    )

    return cluster_price_levels(
        pivots=pivots,
        tolerance=atr * ZONE_ATR_TOLERANCE,
        level_type="RESISTANCE",
    )


def level_distance(
    price: float,
    level: Dict[str, Any],
) -> float:
```

```
    """Distance entre un prix et une zone de support/résistance."""
    price = _finite_number(price, "price")

    lower = _finite_number(
        level["lower"],
        "level.lower",
    )

    upper = _finite_number(
        level["upper"],
        "level.upper",
    )

    if lower > upper:
        raise StrategyError(
            "Les limites de la zone sont inversées."
        )

    if lower <= price <= upper:
        return 0.0

    if price < lower:
        return lower - price

    return price - upper


def find_nearest_level(
    price: float,
    levels: List[Dict[str, Any]],
    level_type: Optional[str] = None,
    direction: Optional[str] = None,
) -> Optional[Dict[str, Any]]:
    """Renvoie le niveau le plus proche respectant les filtres."""
    price = _finite_number(price, "price")

    candidates = []

    for level in levels:
        if not isinstance(level, dict):
            continue

        if (
            level_type is not None
            and level.get("type") != level_type
        ):
            continue

        level_price = _finite_number(
            level.get("price"),
            "level.price",
        )

        if direction == "ABOVE" and level_price <= price:
            continue

        if direction == "BELOW" and level_price >= price:
            continue
```

```
        if direction not in (
            None,
            "ABOVE",
            "BELOW",
        ):
            raise StrategyError(
                "Direction de recherche de niveau invalide."
            )

        candidates.append(level)

    if not candidates:
        return None

    return min(
        candidates,
        key=lambda level: level_distance(
            price,
            level,
        ),
    )


def filter_levels_by_strength(
    levels: List[Dict[str, Any]],
    minimum_touches: int = 2,
) -> List[Dict[str, Any]]:
    """Écarte les zones trop peu testées."""
    if minimum_touches < 1:
        raise StrategyError(
            "Le nombre minimal de touches doit être positif."
        )

    return [
        level
        for level in levels
        if int(level.get("touches", 0))
        >= minimum_touches
          ]
def calculate_linear_regression(
    points: List[Tuple[float, float]],
) -> Tuple[float, float]:
    """Calcule la pente et l'ordonnée à l'origine d'une droite."""
    if len(points) < 2:
        raise StrategyError(
            "Au moins deux points sont nécessaires."
        )

    x_values = [
        _finite_number(point[0], "point.x")
        for point in points
    ]

    y_values = [
        _finite_number(point[1], "point.y")
        for point in points
    ]
```

```
    mean_x = sum(x_values) / len(x_values)
    mean_y = sum(y_values) / len(y_values)

    denominator = sum(
        (x - mean_x) ** 2
        for x in x_values
    )

    if denominator == 0:
        raise StrategyError(
            "Impossible de calculer la pente."
        )

    numerator = sum(
        (x - mean_x) * (y - mean_y)
        for x, y in zip(x_values, y_values)
    )

    slope = numerator / denominator
    intercept = mean_y - slope * mean_x

    if not (
        math.isfinite(slope)
        and math.isfinite(intercept)
    ):
        raise StrategyError(
            "La droite calculée est invalide."
        )

    return slope, intercept


def line_value_at(
    slope: float,
    intercept: float,
    index: float,
) -> float:
    """Calcule la valeur d'une droite à un index donné."""
    slope = _finite_number(slope, "slope")
    intercept = _finite_number(
        intercept,
        "intercept",
    )
    index = _finite_number(index, "index")

    value = slope * index + intercept

    if not math.isfinite(value):
        raise StrategyError(
            "Valeur de trendline non finie."
        )

    return value


def count_trendline_touches(
    candles: List[Dict[str, Any]],
```

```
    pivots: List[Dict[str, Any]],
    slope: float,
    intercept: float,
    tolerance: float,
) -> List[Dict[str, Any]]:
    """Compte les contacts entre pivots et une trendline."""
    normalized = validate_candle_series(
        candles,
        minimum_count=3,
    )

    tolerance = _finite_number(
        tolerance,
        "tolerance",
    )

    if tolerance < 0:
        raise StrategyError(
            "La tolérance ne peut pas être négative."
        )

    touches = []

    for pivot in pivots:
        try:
            index = int(pivot["index"])
            price = _finite_number(
                pivot["price"],
                "pivot.price",
            )
        except (KeyError, TypeError, ValueError, StrategyError):
            continue

        if index < 0 or index >= len(normalized):
            continue

        expected_price = line_value_at(
            slope,
            intercept,
            index,
        )

        distance = abs(
            price - expected_price
        )

        if distance <= tolerance:
            touches.append(
                {
                    "index": index,
                    "epoch": int(
                        normalized[index]["epoch"]
                    ),
                    "price": price,
                    "line_price": expected_price,
                    "distance": distance,
                }
            )
```

```
    return touches


def detect_trendlines(
    candles: List[Dict[str, Any]],
    atr: Optional[float] = None,
    radius: int = 2,
    minimum_touches: int = MIN_TRENDLINE_TOUCHES,
) -> Dict[str, List[Dict[str, Any]]]:
    """
    Détecte des trendlines haussières et baissières.

    Les lignes ne sont retenues que si elles présentent
    suffisamment de contacts avec des pivots.
    """
    normalized = validate_candle_series(
        candles,
        minimum_count=max(
            MIN_CANDLE_COUNT,
            (radius * 2) + 1,
        ),
    )

    if minimum_touches < 3:
        raise StrategyError(
            "Une trendline doit avoir au moins trois touches."
        )

    if atr is None:
        atr = calculate_atr(normalized)

    atr = _finite_number(
        atr,
        "ATR",
    )

    if atr <= 0:
        raise StrategyError(
            "L'ATR doit être positif."
        )

    tolerance = atr * ZONE_ATR_TOLERANCE

    low_pivots = find_pivot_lows(
        normalized,
        radius=radius,
    )

    high_pivots = find_pivot_highs(
        normalized,
        radius=radius,
    )

    result = {
        "uptrend": [],
        "downtrend": [],
    }
```

```
    def build_candidates(
        pivots: List[Dict[str, Any]],
        trend_type: str,
    ) -> List[Dict[str, Any]]:
        candidates = []

        if len(pivots) < minimum_touches:
            return candidates

        pivot_count = len(pivots)

        for start in range(pivot_count - 1):
            for end in range(
                start + 1,
                pivot_count,
            ):
                first = pivots[start]
                last = pivots[end]

                if last["index"] <= first["index"]:
                    continue

                slope = (
                    last["price"] - first["price"]
                ) / (
                    last["index"] - first["index"]
                )

                if trend_type == "UPTREND" and slope <= 0:
                    continue

                if trend_type == "DOWNTREND" and slope >= 0:
                    continue

                intercept = (
                    first["price"]
                    - slope * first["index"]
                )

                touches = count_trendline_touches(
                    normalized,
                    pivots,
                    slope,
                    intercept,
                    tolerance,
                )

                if len(touches) < minimum_touches:
                    continue

                indices = [
                    touch["index"]
                    for touch in touches
                ]

                if len(set(indices)) < minimum_touches:
                    continue
```

```
            current_index = len(normalized) - 1

            current_line_price = line_value_at(
                slope,
                intercept,
                current_index,
            )

            candidate = {
                "type": trend_type,
                "slope": slope,
                "intercept": intercept,
                "start_index": min(indices),
                "end_index": max(indices),
                "touches": len(touches),
                "touch_points": touches,
                "current_value": current_line_price,
                "tolerance": tolerance,
            }

            candidates.append(candidate)

    candidates.sort(
        key=lambda candidate: (
            candidate["touches"],
            candidate["end_index"],
            -abs(candidate["slope"]),
        ),
        reverse=True,
    )

    unique = []
    signatures = set()

    for candidate in candidates:
        signature = (
            round(candidate["slope"], 8),
            round(candidate["intercept"], 6),
        )

        if signature in signatures:
            continue

        signatures.add(signature)
        unique.append(candidate)

        if len(unique) >= 10:
            break

    return unique

result["uptrend"] = build_candidates(
    low_pivots,
    "UPTREND",
)

result["downtrend"] = build_candidates(
```

```
        high_pivots,
        "DOWNTREND",
    )

    LOGGER.debug(
        "Trendlines détectées : haussières=%s, baissières=%s",
        len(result["uptrend"]),
        len(result["downtrend"]),
    )

    return result


def get_nearest_trendline(
    trendlines: Dict[str, List[Dict[str, Any]]],
    current_price: float,
    trend_type: str,
) -> Optional[Dict[str, Any]]:
    """Retourne la trendline valide la plus proche du prix."""
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    if trend_type == "UPTREND":
        key = "uptrend"
    elif trend_type == "DOWNTREND":
        key = "downtrend"
    else:
        raise StrategyError(
            "Type de trendline inconnu."
        )

    candidates = trendlines.get(key, [])

    if not candidates:
        return None

    valid_candidates = [
        line
        for line in candidates
        if int(line.get("touches", 0))
        >= MIN_TRENDLINE_TOUCHES
    ]

    if not valid_candidates:
        return None

    return min(
        valid_candidates,
        key=lambda line: abs(
            current_price - float(
                line["current_value"]
            )
        ),
    )
```

```
def trendline_distance(
    current_price: float,
    trendline: Dict[str, Any],
) -> float:
    """Mesure la distance absolue entre le prix et une trendline."""
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    line_price = _finite_number(
        trendline["current_value"],
        "trendline.current_value",
    )

    return abs(
        current_price - line_price
    )


def is_price_near_trendline(
    current_price: float,
    trendline: Dict[str, Any],
    atr: float,
    maximum_distance_atr: float = MAX_PRICE_DISTANCE_ATR,
) -> bool:
    """Vérifie si le prix approche suffisamment une trendline."""
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    atr = _finite_number(
        atr,
        "ATR",
    )

    maximum_distance_atr = _finite_number(
        maximum_distance_atr,
        "maximum_distance_atr",
    )

    if atr <= 0 or maximum_distance_atr < 0:
        return False

    if int(
        trendline.get("touches", 0)
    ) < MIN_TRENDLINE_TOUCHES:
        return False

    distance = trendline_distance(
        current_price,
        trendline,
    )

    return distance <= (
        atr * maximum_distance_atr
      )
```

```
def determine_market_trend(
    candles: List[Dict[str, Any]],
    short_period: int = 20,
    long_period: int = 50,
) -> Dict[str, Any]:
    """Détermine une tendance simple à partir des moyennes mobiles."""
    normalized = validate_candle_series(
        candles,
        minimum_count=long_period + 1,
    )

    if short_period < 2 or long_period <= short_period:
        raise StrategyError(
            "Les périodes des moyennes mobiles sont invalides."
        )

    closes = [
        candle["close"]
        for candle in normalized
    ]

    short_average = sum(
        closes[-short_period:]
    ) / short_period

    long_average = sum(
        closes[-long_period:]
    ) / long_period

    recent_close = closes[-1]

    if short_average > long_average and recent_close > short_average:
        direction = "BULLISH"

    elif short_average < long_average and recent_close < short_average:
        direction = "BEARISH"

    else:
        direction = "NEUTRAL"

    return {
        "direction": direction,
        "short_average": short_average,
        "long_average": long_average,
        "last_close": recent_close,
    }


def detect_market_structure(
    candles: List[Dict[str, Any]],
    radius: int = 2,
) -> Dict[str, Any]:
    """Analyse les derniers sommets et creux significatifs."""
    normalized = validate_candle_series(
        candles,
        minimum_count=max(
            MIN_CANDLE_COUNT,
            (radius * 2) + 1,
```

```
    ),
)

highs = find_pivot_highs(
    normalized,
    radius=radius,
)

lows = find_pivot_lows(
    normalized,
    radius=radius,
)

last_two_highs = highs[-2:]
last_two_lows = lows[-2:]

higher_high = False
lower_high = False
higher_low = False
lower_low = False

if len(last_two_highs) == 2:
    higher_high = (
        last_two_highs[-1]["price"]
        > last_two_highs[-2]["price"]
    )

    lower_high = (
        last_two_highs[-1]["price"]
        < last_two_highs[-2]["price"]
    )

if len(last_two_lows) == 2:
    higher_low = (
        last_two_lows[-1]["price"]
        > last_two_lows[-2]["price"]
    )

    lower_low = (
        last_two_lows[-1]["price"]
        < last_two_lows[-2]["price"]
    )

if higher_high and higher_low:
    direction = "BULLISH"

elif lower_high and lower_low:
    direction = "BEARISH"

else:
    direction = "RANGE"

return {
    "direction": direction,
    "higher_high": higher_high,
    "lower_high": lower_high,
    "higher_low": higher_low,
    "lower_low": lower_low,
```

```
        "pivot_highs": highs,
        "pivot_lows": lows,
    }


def find_nearest_support(
    current_price: float,
    supports: List[Dict[str, Any]],
) -> Optional[Dict[str, Any]]:
    """Trouve le support situé au niveau ou sous le prix."""
    return find_nearest_level(
        price=current_price,
        levels=supports,
        level_type="SUPPORT",
        direction="BELOW",
    )


def find_nearest_resistance(
    current_price: float,
    resistances: List[Dict[str, Any]],
) -> Optional[Dict[str, Any]]:
    """Trouve la résistance située au niveau ou au-dessus du prix."""
    return find_nearest_level(
        price=current_price,
        levels=resistances,
        level_type="RESISTANCE",
        direction="ABOVE",
    )


def calculate_confluence_score(
    current_price: float,
    atr: float,
    level: Dict[str, Any],
    trendline: Optional[Dict[str, Any]],
    expected_direction: str,
) -> Dict[str, Any]:
    """
    Évalue la proximité entre une zone technique et une trendline.

    Le score exprime une confluence géométrique, pas une probabilité
    de réussite du trade.
    """
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    atr = _finite_number(
        atr,
        "ATR",
    )

    if atr <= 0:
        raise StrategyError(
            "L'ATR doit être positif."
        )
```

```
    if expected_direction not in ("BUY", "SELL"):
        raise StrategyError(
            "La direction attendue doit être BUY ou SELL."
        )

    level_price = _finite_number(
        level["price"],
        "level.price",
    )

    level_type = level.get("type")

    if expected_direction == "BUY" and level_type != "SUPPORT":
        return {
            "score": 0,
            "confluence": False,
            "reasons": [
                "La zone n'est pas un support."
            ],
        }

    if expected_direction == "SELL" and level_type != "RESISTANCE":
        return {
            "score": 0,
            "confluence": False,
            "reasons": [
                "La zone n'est pas une résistance."
            ],
        }

    reasons = []
    score = 0

    distance_to_level = level_distance(
        current_price,
        level,
    )

    if distance_to_level <= atr * MAX_PRICE_DISTANCE_ATR:
        score += 1
        reasons.append(
            "Prix proche de la zone technique."
        )

    touches = int(
        level.get("touches", 0)
    )

    if touches >= 3:
        score += 1
        reasons.append(
            "Zone testée au moins trois fois."
        )

    trendline_distance_value = None

    if trendline is not None:
```

```
        trendline_distance_value = trendline_distance(
            current_price,
            trendline,
        )

        if (
            trendline.get("type")
            == (
                "UPTREND"
                if expected_direction == "BUY"
                else "DOWNTREND"
            )
            and touches >= 1
        ):
            if trendline_distance_value <= atr * MAX_PRICE_DISTANCE_ATR:
                score += 1
                reasons.append(
                    "Trendline compatible avec la direction."
                )

            line_price = _finite_number(
                trendline["current_value"],
                "trendline.current_value",
            )

            level_line_gap = abs(
                line_price - level_price
            )

            if level_line_gap <= atr * ZONE_ATR_TOLERANCE:
                score += 1
                reasons.append(
                    "Trendline proche de la zone technique."
                )

    return {
        "score": score,
        "confluence": score >= 3,
        "reasons": reasons,
        "distance_to_level": distance_to_level,
        "trendline_distance": trendline_distance_value,
        "level_price": level_price,
    }


def build_anticipatory_zone(
    level: Dict[str, Any],
    trendline: Optional[Dict[str, Any]],
    atr: float,
    direction: str,
) -> Dict[str, Any]:
    """Construit une zone de surveillance anticipée."""
    atr = _finite_number(
        atr,
        "ATR",
    )

    if atr <= 0:
```

```
        raise StrategyError(
            "L'ATR doit être positif."
        )

    if direction not in ("BUY", "SELL"):
        raise StrategyError(
            "Direction invalide pour la zone anticipée."
        )

    lower = _finite_number(
        level["lower"],
        "level.lower",
    )

    upper = _finite_number(
        level["upper"],
        "level.upper",
    )

    if lower > upper:
        raise StrategyError(
            "Les limites de la zone sont inversées."
        )

    level_price = _finite_number(
        level["price"],
        "level.price",
    )

    zone_lower = lower
    zone_upper = upper

    if trendline is not None:
        line_price = _finite_number(
            trendline["current_value"],
            "trendline.current_value",
        )

        line_lower = (
            line_price - atr * ZONE_ATR_TOLERANCE
        )

        line_upper = (
            line_price + atr * ZONE_ATR_TOLERANCE
        )

        intersection_lower = max(
            zone_lower,
            line_lower,
        )

        intersection_upper = min(
            zone_upper,
            line_upper,
        )

        if intersection_lower <= intersection_upper:
            zone_lower = intersection_lower
```

```
            zone_upper = intersection_upper

    approach_buffer = atr * ZONE_ATR_TOLERANCE

    if direction == "BUY":
        entry_min = zone_lower - approach_buffer
        entry_max = zone_upper + approach_buffer
    else:
        entry_min = zone_lower - approach_buffer
        entry_max = zone_upper + approach_buffer

    return {
        "direction": direction,
        "level_type": level.get("type"),
        "level_price": level_price,
        "zone_lower": zone_lower,
        "zone_upper": zone_upper,
        "entry_zone_lower": entry_min,
        "entry_zone_upper": entry_max,
        "atr": atr,
        "trendline_value": (
            trendline.get("current_value")
            if trendline is not None
            else None
        ),
        "anticipatory": True,
    }


def is_price_approaching_zone(
    current_price: float,
    zone: Dict[str, Any],
    atr: float,
    maximum_distance_atr: float = MAX_PRICE_DISTANCE_ATR,
) -> bool:
    """Indique si le prix est dans la zone ou s'en approche."""
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    atr = _finite_number(
        atr,
        "ATR",
    )

    maximum_distance_atr = _finite_number(
        maximum_distance_atr,
        "maximum_distance_atr",
    )

    if atr <= 0 or maximum_distance_atr < 0:
        return False

    lower = _finite_number(
        zone["entry_zone_lower"],
        "entry_zone_lower",
    )
```

```
    upper = _finite_number(
        zone["entry_zone_upper"],
        "entry_zone_upper",
    )

    if lower > upper:
        return False

    if lower <= current_price <= upper:
        return True

    if current_price < lower:
        return (lower - current_price) <= (
            atr * maximum_distance_atr
        )

    return (current_price - upper) <= (
        atr * maximum_distance_atr
  )
def calculate_stop_loss(
    direction: str,
    entry_price: float,
    zone: Dict[str, Any],
    atr: float,
    atr_buffer: float = ATR_STOP_BUFFER,
) -> float:
    """Place le Stop Loss au-delà de la zone technique."""
    entry_price = _finite_number(
        entry_price,
        "entry_price",
    )
    atr = _finite_number(atr, "ATR")
    atr_buffer = _finite_number(
        atr_buffer,
        "atr_buffer",
    )

    if entry_price <= 0 or atr <= 0:
        raise StrategyError(
            "Le prix d'entrée et l'ATR doivent être positifs."
        )

    if atr_buffer < 0:
        raise StrategyError(
            "La marge ATR ne peut pas être négative."
        )

    zone_lower = _finite_number(
        zone["zone_lower"],
        "zone_lower",
    )
    zone_upper = _finite_number(
        zone["zone_upper"],
        "zone_upper",
    )

    if zone_lower > zone_upper:
```

```
        raise StrategyError(
            "Les limites de la zone sont inversées."
        )

    buffer = atr * atr_buffer

    if direction == "BUY":
        stop_loss = zone_lower - buffer

        if stop_loss >= entry_price:
            raise StrategyError(
                "Le Stop Loss BUY doit être sous l'entrée."
            )

    elif direction == "SELL":
        stop_loss = zone_upper + buffer

        if stop_loss <= entry_price:
            raise StrategyError(
                "Le Stop Loss SELL doit être au-dessus de l'entrée."
            )

    else:
        raise StrategyError(
            "Direction invalide pour le Stop Loss."
        )

    if stop_loss <= 0:
        raise StrategyError(
            "Le Stop Loss calculé est invalide."
        )

    return stop_loss


def calculate_take_profit(
    direction: str,
    entry_price: float,
    supports: List[Dict[str, Any]],
    resistances: List[Dict[str, Any]],
    minimum_rr: float = MIN_RISK_REWARD,
    stop_loss: Optional[float] = None,
) -> float:
    """Recherche un objectif technique respectant le RR minimal."""
    entry_price = _finite_number(
        entry_price,
        "entry_price",
    )
    minimum_rr = _finite_number(
        minimum_rr,
        "minimum_rr",
    )

    if entry_price <= 0:
        raise StrategyError(
            "Le prix d'entrée doit être positif."
        )
```

```
    if minimum_rr <= 0:
        raise StrategyError(
            "Le ratio risque/rendement doit être positif."
        )

    if stop_loss is None:
        raise StrategyError(
            "Le Stop Loss est obligatoire pour calculer le TP."
        )

    stop_loss = _finite_number(
        stop_loss,
        "stop_loss",
    )

    if direction == "BUY":
        if stop_loss >= entry_price:
            raise StrategyError(
                "Pour un BUY, le SL doit être sous l'entrée."
            )

        risk = entry_price - stop_loss
        candidates = [
            level
            for level in resistances
            if float(level.get("price", 0)) > entry_price
        ]

        candidates.sort(
            key=lambda level: float(level["price"])
        )

        for level in candidates:
            target = float(level["price"])

            if target <= entry_price:
                continue

            reward = target - entry_price

            if reward / risk >= minimum_rr:
                return target

        return entry_price + risk * minimum_rr

    if direction == "SELL":
        if stop_loss <= entry_price:
            raise StrategyError(
                "Pour un SELL, le SL doit être au-dessus de l'entrée."
            )

        risk = stop_loss - entry_price
        candidates = [
            level
            for level in supports
            if float(level.get("price", 0)) < entry_price
        ]
```

```
        candidates.sort(
            key=lambda level: float(level["price"]),
            reverse=True,
        )

        for level in candidates:
            target = float(level["price"])

            if target >= entry_price:
                continue

            reward = entry_price - target

            if reward / risk >= minimum_rr:
                return target

        return max(
            entry_price - risk * minimum_rr,
            entry_price * 0.000001,
        )

    raise StrategyError(
        "Direction invalide pour le Take Profit."
    )


def calculate_risk_reward(
    direction: str,
    entry_price: float,
    stop_loss: float,
    take_profit: float,
) -> float:
    """Calcule le ratio entre le gain potentiel et le risque."""
    entry_price = _finite_number(
        entry_price,
        "entry_price",
    )
    stop_loss = _finite_number(
        stop_loss,
        "stop_loss",
    )
    take_profit = _finite_number(
        take_profit,
        "take_profit",
    )

    if direction == "BUY":
        risk = entry_price - stop_loss
        reward = take_profit - entry_price

    elif direction == "SELL":
        risk = stop_loss - entry_price
        reward = entry_price - take_profit

    else:
        raise StrategyError(
            "Direction invalide pour le calcul du RR."
        )
```

```
    if risk <= 0:
        raise StrategyError(
            "Le risque doit être strictement positif."
        )

    if reward <= 0:
        raise StrategyError(
            "Le gain potentiel doit être strictement positif."
        )

    ratio = reward / risk

    if not math.isfinite(ratio):
        raise StrategyError(
            "Le ratio risque/rendement est invalide."
        )

    return ratio


def validate_trade_levels(
    direction: str,
    entry_price: float,
    stop_loss: float,
    take_profit: float,
    minimum_rr: float = MIN_RISK_REWARD,
) -> Dict[str, Any]:
    """Vérifie la cohérence des niveaux avant de produire un signal."""
    entry_price = _finite_number(
        entry_price,
        "entry_price",
    )
    stop_loss = _finite_number(
        stop_loss
  )
    take_profit = _finite_number(
        take_profit,
        "take_profit",
    )

    if direction == "BUY":
        risk = entry_price - stop_loss
        reward = take_profit - entry_price

    elif direction == "SELL":
        risk = stop_loss - entry_price
        reward = entry_price - take_profit

    else:
        raise StrategyError(
            "Direction invalide pour le calcul du RR."
        )

    if risk <= 0:
        raise StrategyError(
            "Le risque doit être strictement positif."
        )
```

```
    if reward <= 0:
        raise StrategyError(
            "Le gain potentiel doit être strictement positif."
        )

    ratio = reward / risk

    if not math.isfinite(ratio):
        raise StrategyError(
            "Le ratio risque/rendement est invalide."
        )

    return ratio


def validate_trade_levels(
    direction: str,
    entry_price: float,
    stop_loss: float,
    take_profit: float,
    minimum_rr: float = MIN_RISK_REWARD,
) -> Dict[str, Any]:
    """Vérifie la cohérence des niveaux avant de produire un signal."""
    entry_price = _finite_number(
        entry_price,
        "entry_price",
    )
    stop_loss = _finite_number(
        stop_loss,
        "stop_loss",
    )
    take_profit = _finite_number(
        take_profit,
        "take_profit",
    )
    minimum_rr = _finite_number(
        minimum_rr,
        "minimum_rr",
    )

    if min(entry_price, stop_loss, take_profit) <= 0:
        raise StrategyError(
            "Les niveaux de prix doivent être positifs."
        )

    ratio = calculate_risk_reward(
        direction=direction,
        entry_price=entry_price,
        stop_loss=stop_loss,
        take_profit=take_profit,
    )

    if ratio < minimum_rr:
        raise StrategyError(
            f"RR insuffisant : {ratio:.2f}, "
            f"minimum requis : {minimum_rr:.2f}."
        )
```

```
    return {
        "valid": True,
        "direction": direction,
        "entry": entry_price,
        "stop_loss": stop_loss,
        "take_profit": take_profit,
        "risk_reward": ratio,
        "minimum_rr": minimum_rr,
    }


def calculate_entry_price(
    direction: str,
    zone: Dict[str, Any],
    current_price: float,
) -> float:
    """Détermine un prix indicatif dans la zone d'entrée."""
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    if current_price <= 0:
        raise StrategyError(
            "Le prix actuel doit être positif."
        )

    lower = _finite_number(
        zone["entry_zone_lower"],
        "entry_zone_lower",
    )
    upper = _finite_number(
        zone["entry_zone_upper"],
        "entry_zone_upper",
    )

    if lower > upper:
        raise StrategyError(
            "Les limites de la zone d'entrée sont inversées."
        )

    if direction not in ("BUY", "SELL"):
        raise StrategyError(
            "Direction invalide pour l'entrée."
        )

    if lower <= current_price <= upper:
        return current_price

    if current_price < lower:
        return lower

    return upper


def check_minimum_rr(
    direction: str,
```

```
    entry_price: float,
    stop_loss: float,
    take_profit: float,
    minimum_rr: float = MIN_RISK_REWARD,
) -> bool:
    """Retourne True uniquement si le RR minimal est respecté."""
    try:
        result = validate_trade_levels(
            direction=direction,
            entry_price=entry_price,
            stop_loss=stop_loss,
            take_profit=take_profit,
            minimum_rr=minimum_rr,
        )

        return bool(result["valid"])

    except (StrategyError, TypeError, ValueError):
        return False
def analyze_d1_direction(
    d1_candles: List[Dict[str, Any]],
) -> Dict[str, Any]:
    """Détermine la direction générale à partir du Daily."""
    candles = validate_candle_series(
        d1_candles,
        minimum_count=TIMEFRAME_REQUIREMENTS["D1"],
    )

    trend = determine_market_trend(candles)
    structure = detect_market_structure(candles)

    if (
        trend["direction"] == "BULLISH"
        and structure["direction"] == "BULLISH"
    ):
        direction = "BULLISH"
    elif (
        trend["direction"] == "BEARISH"
        and structure["direction"] == "BEARISH"
    ):
        direction = "BEARISH"
    else:
        direction = "NEUTRAL"

    return {
        "timeframe": "D1",
        "direction": direction,
        "trend": trend,
        "structure": structure,
        "candles_count": len(candles),
    }


def analyze_h4_structure(
    h4_candles: List[Dict[str, Any]],
) -> Dict[str, Any]:
    """Analyse la structure principale sur H4."""
    candles = validate_candle_series(
```

```
        h4_candles,
        minimum_count=TIMEFRAME_REQUIREMENTS["H4"],
    )

    atr = calculate_atr(candles)
    trend = determine_market_trend(candles)
    structure = detect_market_structure(candles)

    supports = find_supports(
        candles,
        atr=atr,
    )

    resistances = find_resistances(
        candles,
        atr=atr,
    )

    trendlines = detect_trendlines(
        candles,
        atr=atr,
    )

    return {
        "timeframe": "H4",
        "atr": atr,
        "trend": trend,
        "structure": structure,
        "supports": supports,
        "resistances": resistances,
        "trendlines": trendlines,
        "candles_count": len(candles),
    }


def analyze_h2_confirmation(
    h2_candles: List[Dict[str, Any]],
) -> Dict[str, Any]:
    """Analyse la tendance intermédiaire sur H2."""
    candles = validate_candle_series(
        h2_candles,
        minimum_count=TIMEFRAME_REQUIREMENTS["H2"],
    )

    trend = determine_market_trend(candles)
    structure = detect_market_structure(candles)
    atr = calculate_atr(candles)

    return {
        "timeframe": "H2",
        "direction": trend["direction"],
        "trend": trend,
        "structure": structure,
        "atr": atr,
        "candles_count": len(candles),
    }
```

```
def analyze_h1_entry_context(
    h1_candles: List[Dict[str, Any]],
) -> Dict[str, Any]:
    """Prépare les données H1 pour affiner la zone d'entrée."""
    candles = validate_candle_series(
        h1_candles,
        minimum_count=TIMEFRAME_REQUIREMENTS["H1"],
    )

    atr = calculate_atr(candles)
    trend = determine_market_trend(candles)
    structure = detect_market_structure(candles)

    supports = find_supports(
        candles,
        atr=atr,
    )

    resistances = find_resistances(
        candles,
        atr=atr,
    )

    trendlines = detect_trendlines(
        candles,
        atr=atr,
    )

    return {
        "timeframe": "H1",
        "atr": atr,
        "trend": trend,
        "structure": structure,
        "supports": supports,
        "resistances": resistances,
        "trendlines": trendlines,
        "candles_count": len(candles),
    }


def check_timeframe_alignment(
    d1_analysis: Dict[str, Any],
    h4_analysis: Dict[str, Any],
    h2_analysis: Dict[str, Any],
) -> Dict[str, Any]:
    """Vérifie l'accord entre Daily, H4 et H2."""
    d1_direction = d1_analysis.get(
        "direction",
        "NEUTRAL",
    )

    h4_direction = h4_analysis.get(
        "structure",
        {},
    ).get(
        "direction",
        "RANGE",
    )
```

```
    h2_direction = h2_analysis.get(
        "direction",
        "NEUTRAL",
    )

    if d1_direction == "BULLISH":
        expected = "BUY"
        h4_aligned = h4_direction == "BULLISH"
        h2_aligned = h2_direction == "BULLISH"

    elif d1_direction == "BEARISH":
        expected = "SELL"
        h4_aligned = h4_direction == "BEARISH"
        h2_aligned = h2_direction == "BEARISH"

    else:
        return {
            "aligned": False,
            "direction": None,
            "score": 0,
            "reasons": [
                "La direction D1 n'est pas suffisamment claire."
            ],
        }

    score = 1
    reasons = [
        f"Direction générale D1 : {d1_direction}.",
    ]

    if h4_aligned:
        score += 1
        reasons.append(
            "La structure H4 est alignée."
        )
    else:
        reasons.append(
            "La structure H4 n'est pas alignée."
        )

    if h2_aligned:
        score += 1
        reasons.append(
            "La tendance H2 est alignée."
        )
    else:
        reasons.append(
            "La tendance H2 n'est pas alignée."
        )

    return {
        "aligned": h4_aligned and h2_aligned,
        "direction": expected,
        "score": score,
        "reasons": reasons,
    }
```

```
def detect_bos(
    candles: List[Dict[str, Any]],
    radius: int = 2,
) -> Dict[str, Any]:
    """Détecte une éventuelle cassure de structure."""
    normalized = validate_candle_series(
        candles,
        minimum_count=max(
            MIN_CANDLE_COUNT,
            (radius * 2) + 1,
        ),
    )

    highs = find_pivot_highs(
        normalized,
        radius=radius,
    )

    lows = find_pivot_lows(
        normalized,
        radius=radius,
    )

    last_close = normalized[-1]["close"]

    bullish_bos = False
    bearish_bos = False
    broken_level = None

    if highs:
        latest_high = highs[-1]

        if (
            last_close > latest_high["price"]
            and normalized[-2]["close"]
            <= latest_high["price"]
        ):
            bullish_bos = True
            broken_level = latest_high["price"]

    if lows:
        latest_low = lows[-1]

        if (
            last_close < latest_low["price"]
            and normalized[-2]["close"]
            >= latest_low["price"]
        ):
            bearish_bos = True
            broken_level = latest_low["price"]

    if bullish_bos and not bearish_bos:
        direction = "BULLISH"
    elif bearish_bos and not bullish_bos:
        direction = "BEARISH"
    else:
        direction = "NONE"
```

```
    return {
        "detected": direction != "NONE",
        "direction": direction,
        "broken_level": broken_level,
        "last_close": last_close,
    }


def detect_displacement(
    candles: List[Dict[str, Any]],
    atr: Optional[float] = None,
    minimum_body_ratio: float = 0.55,
    minimum_range_atr: float = 0.80,
) -> Dict[str, Any]:
    """Détecte une bougie impulsive relativement à l'ATR."""
    normalized = validate_candle_series(
        candles,
        minimum_count=ATR_PERIOD + 1,
    )

    if atr is None:
        atr = calculate_atr(normalized)

    atr = _finite_number(atr, "ATR")

    if atr <= 0:
        raise StrategyError(
            "ATR invalide pour détecter le displacement."
        )

    candle = normalized[-1]

    candle_range = (
        candle["high"] - candle["low"]
    )

    body_ratio = candle_body_ratio(candle)

    if candle_range <= 0:
        return {
            "detected": False,
            "direction": "NONE",
            "range": candle_range,
            "body_ratio": body_ratio,
            "range_atr": 0.0,
        }

    range_atr = candle_range / atr

    detected = (
        body_ratio >= minimum_body_ratio
        and range_atr >= minimum_range_atr
    )

    direction = (
        candle_direction(candle)
        if detected
```

```
        else "NONE"
    )

    if direction == "DOJI":
        direction = "NONE"
        detected = False

    return {
        "detected": detected,
        "direction": direction,
        "range": candle_range,
        "body_ratio": body_ratio,
        "range_atr": range_atr,
        "candle": candle,
}
def detect_order_blocks(
    candles: List[Dict[str, Any]],
    lookback: int = 40,
    search_back: int = 5,
    atr: Optional[float] = None,
) -> List[Dict[str, Any]]:
    """
    Recherche les Order Blocks potentiels avant un déplacement.

    Un Order Block reste une zone candidate : sa présence seule
    ne constitue pas un signal d'entrée.
    """
    normalized = validate_candle_series(
        candles,
        minimum_count=max(
            MIN_CANDLE_COUNT,
            search_back + 2,
        ),
    )

    if lookback < 1 or search_back < 1:
        raise StrategyError(
            "Les paramètres de recherche des Order Blocks "
            "doivent être positifs."
        )

    if atr is None:
        atr = calculate_atr(normalized)

    atr = _finite_number(
        atr,
        "ATR",
    )

    if atr <= 0:
        raise StrategyError(
            "L'ATR doit être positif."
        )

    start_index = max(
        1,
        len(normalized) - lookback,
    )
```

```
    order_blocks = []

    for displacement_index in range(
        start_index,
        len(normalized),
    ):
        displacement_candle = normalized[
            displacement_index
        ]

        candle_range = (
            displacement_candle["high"]
            - displacement_candle["low"]
        )

        if candle_range <= 0:
            continue

        body_ratio = candle_body_ratio(
            displacement_candle
        )

        if (
            body_ratio < 0.55
            or candle_range < atr * 0.80
        ):
            continue

        displacement_direction = candle_direction(
            displacement_candle
        )

        if displacement_direction == "DOJI":
            continue

        search_start = max(
            0,
            displacement_index - search_back,
        )

        candidate_index = None

        for index in range(
            displacement_index - 1,
            search_start - 1,
            -1,
        ):
            previous_candle = normalized[index]
            previous_direction = candle_direction(
                previous_candle
            )

            if (
                displacement_direction == "BULLISH"
                and previous_direction == "BEARISH"
            ):
                candidate_index = index
```

```
            break

        if (
            displacement_direction == "BEARISH"
            and previous_direction == "BULLISH"
        ):
            candidate_index = index
            break

    if candidate_index is None:
        continue

    block = normalized[candidate_index]

    if displacement_direction == "BULLISH":
        block_type = "BULLISH"
    else:
        block_type = "BEARISH"

    order_blocks.append(
        {
            "type": block_type,
            "index": candidate_index,
            "epoch": block["epoch"],
            "open": block["open"],
            "high": block["high"],
            "low": block["low"],
            "close": block["close"],
            "zone_lower": block["low"],
            "zone_upper": block["high"],
            "displacement_index": displacement_index,
            "displacement_epoch": (
                displacement_candle["epoch"]
            ),
            "displacement_range_atr": (
                candle_range / atr
            ),
            "body_ratio": body_ratio,
            "entry_reference": block["open"],
            "status": "CANDIDATE",
        }
    )

unique_blocks = []
seen = set()

for block in sorted(
    order_blocks,
    key=lambda item: item["displacement_index"],
):
    key = (
        block["type"],
        block["index"],
    )

    if key in seen:
        continue
```

```
        seen.add(key)
        unique_blocks.append(block)

    return unique_blocks


def is_price_inside_order_block(
    current_price: float,
    order_block: Dict[str, Any],
) -> bool:
    """Vérifie si le prix se trouve dans un Order Block."""
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    lower = _finite_number(
        order_block["zone_lower"],
        "zone_lower",
    )

    upper = _finite_number(
        order_block["zone_upper"],
        "zone_upper",
    )

    if lower > upper:
        raise StrategyError(
            "Les limites de l'Order Block sont inversées."
        )

    return lower <= current_price <= upper


def is_price_near_order_block(
    current_price: float,
    order_block: Dict[str, Any],
    atr: float,
    maximum_distance_atr: float = MAX_PRICE_DISTANCE_ATR,
) -> bool:
    """Vérifie si le prix approche d'un Order Block."""
    current_price = _finite_number(
        current_price,
        "current_price",
    )

    atr = _finite_number(
        atr,
        "ATR",
    )

    maximum_distance_atr = _finite_number(
        maximum_distance_atr,
        "maximum_distance_atr",
    )

    if atr <= 0 or maximum_distance_atr < 0:
        return False
```

```
    lower = _finite_number(
        order_block["zone_lower"],
        "zone_lower",
    )

    upper = _finite_number(
        order_block["zone_upper"],
        "zone_upper",
    )

    if lower > upper:
        return False

    if lower <= current_price <= upper:
        return True

    if current_price < lower:
        distance = lower - current_price
    else:
        distance = current_price - upper

    return distance <= (
        atr * maximum_distance_atr
    )


def filter_order_blocks_by_direction(
    order_blocks: List[Dict[str, Any]],
    direction: str,
) -> List[Dict[str, Any]]:
    """Conserve les Order Blocks compatibles avec la direction."""
    if direction == "BUY":
        expected_type = "BULLISH"
    elif direction == "SELL":
        expected_type = "BEARISH"
    else:
        raise StrategyError(
            "Direction invalide pour filtrer les Order Blocks."
        )

    return [
        block
        for block in order_blocks
        if block.get("type") == expected_type
    ]


def select_nearest_order_block(
    current_price: float,
    order_blocks: List[Dict[str, Any]],
    direction: str,
    atr: float,
    maximum_distance_atr: float = MAX_PRICE_DISTANCE_ATR,
) -> Optional[Dict[str, Any]]:
    """Sélectionne l'Order Block compatible le plus proche."""
    current_price = _finite_number(
        current_price,
```

```
        "current_price",
    )

    atr = _finite_number(
        atr,
        "ATR",
    )

    if current_price <= 0 or atr <= 0:
        return None

    candidates = filter_order_blocks_by_direction(
        order_blocks,
        direction,
    )

    nearby = [
        block
        for block in candidates
        if is_price_near_order_block(
            current_price=current_price,
            order_block=block,
            atr=atr,
            maximum_distance_atr=maximum_distance_atr,
        )
    ]

    if not nearby:
        return None

    def distance_to_block(
        block: Dict[str, Any],
    ) -> float:
        return level_distance(
            current_price,
            {
                "lower": block["zone_lower"],
                "upper": block["zone_upper"],
            },
        )

    nearby.sort(
        key=distance_to_block
    )

    return nearby[0]


def detect_order_block_retest(
    candles: List[Dict[str, Any]],
    order_block: Dict[str, Any],
    direction: str,
) -> Dict[str, Any]:
    """Analyse si les dernières bougies revisitent un Order Block."""
    normalized = validate_candle_series(
        candles,
        minimum_count=3,
    )
```

```
    if direction not in ("BUY", "SELL"):
        raise StrategyError(
            "Direction invalide pour le retest."
        )

    lower = _finite_number(
        order_block["zone_lower"],
        "zone_lower",
    )

    upper = _finite_number(
        order_block["zone_upper"],
        "zone_upper",
    )

    last_candle = normalized[-1]

    overlaps = (
        last_candle["low"] <= upper
        and last_candle["high"] >= lower
    )

    if direction == "BUY":
        reaction = (
            overlaps
            and last_candle["close"] > last_candle["open"]
        )
    else:
        reaction = (
            overlaps
            and last_candle["close"] < last_candle["open"]
        )

    return {
        "retested": overlaps,
        "reaction_detected": reaction,
        "direction": direction,
        "last_close": last_candle["close"],
        "zone_lower": lower,
        "zone_upper": upper,
    }


def summarize_order_block(
    order_block: Dict[str, Any],
) -> Dict[str, Any]:
    """Prépare les informations essentielles d'un Order Block."""
    return {
        "type": order_block.get("type"),
        "zone_lower": order_block.get("zone_lower"),
        "zone_upper": order_block.get("zone_upper"),
        "entry_reference": order_block.get("entry_reference"),
        "displacement_range_atr": (
            order_block.get("displacement_range_atr")
        ),
        "status": order_block.get(
            "status",
```

```
            "CANDIDATE",
        ),
}
def detect_liquidity_sweep(
    candles: List[Dict[str, Any]],
    radius: int = 2,
    lookback: int = 20,
) -> Dict[str, Any]:
    """Détecte un balayage potentiel des sommets ou des creux."""
    normalized = validate_candle_series(
        candles,
        minimum_count=max(
            MIN_CANDLE_COUNT,
            lookback + 1,
            (radius * 2) + 1,
        ),
    )

    if radius < 1 or lookback < 2:
        raise StrategyError(
            "Paramètres invalides pour le balayage de liquidité."
        )

    current = normalized[-1]

    historical = normalized[
        -(lookback + 1):-1
    ]

    if not historical:
        return {
            "detected": False,
            "direction": "NONE",
            "level": None,
        }

    previous_high = max(
        candle["high"]
        for candle in historical
    )

    previous_low = min(
        candle["low"]
        for candle in historical
    )

    swept_high = (
        current["high"] > previous_high
        and current["close"] < previous_high
    )

    swept_low = (
        current["low"] < previous_low
        and current["close"] > previous_low
    )

    if swept_low and not swept_high:
        direction = "BUY"
```

```
        level = previous_low

    elif swept_high and not swept_low:
        direction = "SELL"
        level = previous_high

    else:
        direction = "NONE"
        level = None

    return {
        "detected": direction != "NONE",
        "direction": direction,
        "level": level,
        "swept_high": swept_high,
        "swept_low": swept_low,
        "current_high": current["high"],
        "current_low": current["low"],
        "current_close": current["close"],
    }


def detect_rejection_candle(
    candle: Dict[str, Any],
    direction: str,
    minimum_wick_ratio: float = 0.40,
) -> Dict[str, Any]:
    """Recherche une bougie de rejet compatible avec la direction."""
    normalized = _validate_candle(candle)

    if direction not in ("BUY", "SELL"):
        raise StrategyError(
            "Direction invalide pour la bougie de rejet."
        )

    if not 0 <= minimum_wick_ratio <= 1:
        raise StrategyError(
            "Le ratio de mèche doit être compris entre 0 et 1."
        )

    candle_range = (
        normalized["high"] - normalized["low"]
    )

    if candle_range <= 0:
        return {
            "detected": False,
            "direction": direction,
            "wick_ratio": 0.0,
        }

    if direction == "BUY":
        wick = candle_lower_wick(normalized)

        compatible_close = (
            normalized["close"] >= normalized["open"]
        )
```

```
    else:
        wick = candle_upper_wick(normalized)

        compatible_close = (
            normalized["close"] <= normalized["open"]
        )

    wick_ratio = wick / candle_range

    detected = (
        wick_ratio >= minimum_wick_ratio
        and compatible_close
    )

    return {
        "detected": detected,
        "direction": direction,
        "wick_ratio": wick_ratio,
        "candle_range": candle_range,
        "wick_size": wick,
        "candle": normalized,
    }


def detect_engulfing(
    candles: List[Dict[str, Any]],
) -> Dict[str, Any]:
    """Détecte un éventuel avalement haussier ou baissier."""
    normalized = validate_candle_series(
        candles,
        minimum_count=2,
    )

    previous = normalized[-2]
    current = normalized[-1]

    previous_bullish = (
        previous["close"] > previous["open"]
    )

    previous_bearish = (
        previous["close"] < previous["open"]
    )

    current_bullish = (
        current["close"] > current["open"]
    )

    current_bearish = (
        current["close"] < current["open"]
    )

    bullish_engulfing = (
        previous_bearish
        and current_bullish
        and current["open"] <= previous["close"]
        and current["close"] >= previous["open"]
    )
```

```
    bearish_engulfing = (
        previous_bullish
        and current_bearish
        and current["open"] >= previous["close"]
        and current["close"] <= previous["open"]
    )

    if bullish_engulfing and not bearish_engulfing:
        direction = "BUY"
    elif bearish_engulfing and not bullish_engulfing:
        direction = "SELL"
    else:
        direction = "NONE"

    return {
        "detected": direction != "NONE",
        "direction": direction,
        "bullish_engulfing": bullish_engulfing,
        "bearish_engulfing": bearish_engulfing,
    }


def evaluate_candle_confirmation(
    candles: List[Dict[str, Any]],
    direction: str,
) -> Dict[str, Any]:
    """Combine les principaux indices de confirmation en bougies."""
    normalized = validate_candle_series(
        candles,
        minimum_count=3,
    )

    if direction not in ("BUY", "SELL"):
        raise StrategyError(
            "Direction invalide pour la confirmation."
        )

    last_candle = normalized[-1]

    rejection = detect_rejection_candle(
        last_candle,
        direction,
    )

    engulfing = detect_engulfing(
        normalized[-2:],
    )

    sweep = detect_liquidity_sweep(
        normalized,
    )

    sweep_aligned = (
        sweep["detected"]
        and sweep["direction"] == direction
    )
```

```
    engulfing_aligned = (
        engulfing["detected"]
        and engulfing["direction"] == direction
    )

    score = sum(
        (
            rejection["detected"],
            engulfing_aligned,
            sweep_aligned,
        )
    )

    return {
        "direction": direction,
        "score": score,
        "rejection": rejection,
        "engulfing": engulfing,
        "liquidity_sweep": sweep,
        "confirmed": score >= 1,
    }


def find_recent_doji(
    candles: List[Dict[str, Any]],
    lookback: int = 10,
    maximum_body_ratio: float = 0.10,
) -> Optional[Dict[str, Any]]:
    """Recherche le doji récent le plus proche de la fin de série."""
    normalized = validate_candle_series(
        candles,
        minimum_count=max(lookback, 2),
    )

    if lookback < 1:
        raise StrategyError(
            "La période de recherche doit être positive."
        )

    recent = normalized[-lookback:]

    for index in range(
        len(recent) - 1,
        -1,
        -1,
    ):
        candle = recent[index]

        if is_doji(
            candle,
            maximum_body_ratio=maximum_body_ratio,
        ):
            return {
                "index": len(normalized) - lookback + index,
                "epoch": candle["epoch"],
                "open": candle["open"],
                "high": candle["high"],
                "low": candle["low"],
```

```
                "close": candle["close"],
                "upper_wick": candle_upper_wick(candle),
                "lower_wick": candle_lower_wick(candle),
                "body_ratio": candle_body_ratio(candle),
            }

    return None


def calculate_doji_wick_entry(
    doji: Dict[str, Any],
    direction: str,
) -> float:
    """Propose un niveau d'entrée de référence sur la mèche du doji."""
    if direction not in ("BUY", "SELL"):
        raise StrategyError(
            "Direction invalide pour l'entrée sur doji."
        )

    normalized = _validate_candle(doji)

    if not is_doji(normalized):
        raise StrategyError(
            "La bougie fournie ne satisfait pas le critère doji."
        )

    if direction == "BUY":
        entry = normalized["low"]
    else:
        entry = normalized["high"]

    if entry <= 0:
        raise StrategyError(
            "Le niveau d'entrée du doji est invalide."
        )

    return entry
def analyze_timeframe_consistency(
    d1_analysis: Dict[str, Any],
    h4_analysis: Dict[str, Any],
    h2_analysis: Dict[str, Any],
    h1_analysis: Dict[str, Any],
) -> Dict[str, Any]:
    """
    Vérifie la cohérence entre D1, H4, H2 et H1.
    D1 définit le contexte principal.
    H4 définit la structure.
    H2 vérifie l'alignement.
    H1 affine la zone d'entrée.
    """
    d1_direction = d1_analysis.get("direction", "NEUTRAL")
    h4_direction = h4_analysis.get("structure", {}).get(
        "direction", "RANGE"
    )
    h2_direction = h2_analysis.get("direction", "NEUTRAL")
    h1_direction = h1_analysis.get("direction", "NEUTRAL")

    directions = {
```

```
        "D1": d1_direction,
        "H4": h4_direction,
        "H2": h2_direction,
        "H1": h1_direction,
    }

    bullish_votes = sum(
        1 for value in directions.values()
        if value in ("BULLISH", "BUY")
    )
    bearish_votes = sum(
        1 for value in directions.values()
        if value in ("BEARISH", "SELL")
    )

    if bullish_votes >= 3 and d1_direction in ("BULLISH", "BUY"):
        overall_direction = "BUY"
    elif bearish_votes >= 3 and d1_direction in ("BEARISH", "SELL"):
        overall_direction = "SELL"
    else:
        overall_direction = "NEUTRAL"

    return {
        "direction": overall_direction,
        "timeframes": directions,
        "bullish_votes": bullish_votes,
        "bearish_votes": bearish_votes,
        "aligned": overall_direction != "NEUTRAL",
    }


def get_directional_levels(
    direction: str,
    supports: List[Dict[str, Any]],
    resistances: List[Dict[str, Any]],
    current_price: float,
) -> Dict[str, Optional[Dict[str, Any]]]:
    """
    Sélectionne les niveaux techniques situés du bon côté du prix.
    BUY : support sous le prix et résistance au-dessus.
    SELL : résistance au-dessus du prix et support en dessous.
    """
    if not _finite_number(current_price):
        return {
            "entry_level": None,
            "target_level": None,
            "opposite_level": None,
        }

    valid_supports = [
        level for level in supports
        if isinstance(level, dict)
        and _finite_number(level.get("price"))
    ]

    valid_resistances = [
        level for level in resistances
        if isinstance(level, dict)
```

```
        and _finite_number(level.get("price"))
    ]

    supports_below = [
        level for level in valid_supports
        if float(level["price"]) <= current_price
    ]

    resistances_above = [
        level for level in valid_resistances
        if float(level["price"]) >= current_price
    ]

    nearest_support = (
        min(
            supports_below,
            key=lambda item: current_price - float(item["price"]),
        )
        if supports_below else None
    )

    nearest_resistance = (
        min(
            resistances_above,
            key=lambda item: float(item["price"]) - current_price,
        )
        if resistances_above else None
    )

    if direction == "BUY":
        return {
            "entry_level": nearest_support,
            "target_level": nearest_resistance,
            "opposite_level": nearest_resistance,
        }

    if direction == "SELL":
        return {
            "entry_level": nearest_resistance,
            "target_level": nearest_support,
            "opposite_level": nearest_support,
        }

    return {
        "entry_level": None,
        "target_level": None,
        "opposite_level": None,
    }


def calculate_zone_width(
    atr: float,
    tolerance: float = ZONE_ATR_TOLERANCE,
) -> float:
    """Calcule une largeur de zone adaptée à la volatilité."""
    if not _finite_number(atr) or atr <= 0:
        return 0.0
```

```
    if not _finite_number(tolerance) or tolerance <= 0:
        return 0.0

    return float(atr) * float(tolerance)


def price_is_inside_range(
    price: float,
    lower_bound: float,
    upper_bound: float,
) -> bool:
    """Indique si le prix appartient à une zone définie."""
    if not all(
        _finite_number(value)
        for value in (price, lower_bound, upper_bound)
    ):
        return False

    lower = min(float(lower_bound), float(upper_bound))
    upper = max(float(lower_bound), float(upper_bound))

    return lower <= float(price) <= upper


def distance_to_range(
    price: float,
    lower_bound: float,
    upper_bound: float,
) -> float:
    """Retourne la distance du prix à une zone."""
    if not all(
        _finite_number(value)
        for value in (price, lower_bound, upper_bound)
    ):
        return float("inf")

    lower = min(float(lower_bound), float(upper_bound))
    upper = max(float(lower_bound), float(upper_bound))
    current = float(price)

    if lower <= current <= upper:
        return 0.0

    if current < lower:
        return lower - current

    return current - upper


def validate_market_snapshot(
    snapshot: Dict[str, Any],
) -> Tuple[bool, List[str]]:
    """
    Vérifie qu'un instantané de marché contient les données
    nécessaires avant toute analyse.
    """
    errors = []
```

```
    if not isinstance(snapshot, dict):
        return False, ["L'instantané de marché doit être un dictionnaire."]

    if not _finite_number(snapshot.get("current_price"), "current_price"):
        errors.append("Prix actuel absent ou invalide.")

    timeframe_data = snapshot.get("timeframes")

    if not isinstance(timeframe_data, dict):
        errors.append("Données des unités de temps absentes.")
        return False, errors

    for timeframe, required_count in TIMEFRAME_REQUIREMENTS.items():
        candles = timeframe_data.get(timeframe)

        if not isinstance(candles, list):
            errors.append(
                f"{timeframe} : données de bougies absentes."
            )
            continue

        if len(candles) < required_count:
            errors.append(
                f"{timeframe} : {len(candles)} bougies reçues, "
                f"{required_count} nécessaires."
            )
            continue

        try:
            validate_candle_series(
                candles,
                minimum_count=required_count,
            )
        except (ValueError, TypeError, KeyError) as exc:
            errors.append(f"{timeframe} : données invalides ({exc}).")

    return len(errors) == 0, errors


def normalize_signal_direction(direction: Any) -> str:
    """Normalise les différentes formes de direction."""
    if not isinstance(direction, str):
        return "NEUTRAL"

    value = direction.strip().upper()

    if value in ("BUY", "BULLISH", "LONG", "HAUSSIER"):
        return "BUY"

    if value in ("SELL", "BEARISH", "SHORT", "BAISSIER"):
        return "SELL"

    return "NEUTRAL"


def safe_float(
    value: Any,
    default: Optional[float] = None,
```

```
) -> Optional[float]:
    """Convertit une valeur en nombre sans propager d'exception."""
    try:
        result = float(value)

        if math.isfinite(result):
            return result

    except (TypeError, ValueError, OverflowError):
        pass

    return default


def build_analysis_error(
    symbol: str,
    reason: str,
    details: Optional[List[str]] = None,
) -> Dict[str, Any]:
    """Construit une réponse standard lorsqu'une analyse échoue."""
    return {
        "symbol": symbol,
        "symbol_name": get_symbol_name(symbol),
        "status": "ERROR",
        "signal": None,
        "direction": "NEUTRAL",
        "reason": str(reason),
        "details": details or [],
    }


def build_no_signal_result(
    symbol: str,
    reason: str,
    current_price: Optional[float] = None,
    details: Optional[Dict[str, Any]] = None,
) -> Dict[str, Any]:
    """Construit un résultat valide lorsqu'aucune entrée n'est retenue."""
    return {
        "symbol": symbol,
        "symbol_name": get_symbol_name(symbol),
        "status": "NO_SIGNAL",
        "signal": None,
        "direction": "NEUTRAL",
        "current_price": safe_float(current_price),
        "reason": str(reason),
        "details": details or {},
            }
def analyze_symbol(
    symbol: str,
    snapshot: Dict[str, Any],
) -> Dict[str, Any]:
    """
    Analyse complète d'un indice.

    Principes :
    - D1 : contexte directionnel.
    - H4 : structure, supports, résistances et trendlines.
```

```
- H2 : filtre de confirmation.
- H1 : précision de l'entrée.
- Entrée anticipée sur une confluence technique.
- Aucun ordre n'est exécuté par cette fonction.
"""
symbol = str(symbol).strip()

valid, errors = validate_market_snapshot(snapshot)

if not valid:
    return build_analysis_error(
        symbol,
        "Instantané de marché incomplet ou invalide.",
        errors,
    )

current_price = safe_float(snapshot.get("current_price"))

if current_price is None or current_price <= 0:
    return build_analysis_error(
        symbol,
        "Le prix actuel est invalide.",
    )

timeframe_data = snapshot["timeframes"]

try:
    d1_candles = timeframe_data["D1"]
    h4_candles = timeframe_data["H4"]
    h2_candles = timeframe_data["H2"]
    h1_candles = timeframe_data["H1"]

    d1_analysis = analyze_d1_direction(d1_candles)
    h4_analysis = analyze_h4_structure(h4_candles)
    h2_analysis = analyze_h2_confirmation(h2_candles)
    h1_analysis = analyze_h1_entry_context(h1_candles)

except (ValueError, TypeError, KeyError, IndexError) as exc:
    logger.exception(
        "Erreur pendant l'analyse de %s : %s",
        symbol,
        exc,
    )
    return build_analysis_error(
        symbol,
        "Erreur pendant l'analyse technique.",
        [str(exc)],
    )

consistency = analyze_timeframe_consistency(
    d1_analysis,
    h4_analysis,
    h2_analysis,
    h1_analysis,
)

direction = consistency["direction"]
```

```
    if direction == "NEUTRAL":
        return build_no_signal_result(
            symbol,
            "Les unités D1, H4, H2 et H1 ne présentent pas "
            "un alignement directionnel suffisant.",
            current_price,
            {
                "timeframe_consistency": consistency,
                "d1": d1_analysis,
                "h4": h4_analysis,
                "h2": h2_analysis,
                "h1": h1_analysis,
            },
        )

    try:
        h4_supports = h4_analysis.get("supports", [])
        h4_resistances = h4_analysis.get("resistances", [])
        h4_trendlines = h4_analysis.get("trendlines", {})
        h4_atr = safe_float(h4_analysis.get("atr"), 0.0) or 0.0

        if h4_atr <= 0:
            return build_no_signal_result(
                symbol,
                "ATR H4 absent ou invalide.",
                current_price,
            )

        h4_structure = h4_analysis.get("structure", {})
        structure_direction = normalize_signal_direction(
            h4_structure.get("direction", "NEUTRAL")
        )

        if structure_direction != direction:
            return build_no_signal_result(
                symbol,
                "La structure H4 ne confirme pas la direction principale.",
                current_price,
                {
                    "direction": direction,
                    "h4_structure": h4_structure,
                },
            )

        supports = find_supports(h4_candles)
        resistances = find_resistances(h4_candles)

        if not supports or not resistances:
            return build_no_signal_result(
                symbol,
                "Niveaux H4 insuffisants pour établir une confluence.",
                current_price,
            )

        levels = get_directional_levels(
            direction,
            supports,
            resistances,
```

```
            current_price,
        )

        trendline = get_nearest_trendline(
            current_price,
            h4_trendlines,
        )

        order_blocks = detect_order_blocks(
            h4_candles,
            lookback=40,
            search_back=5,
            atr=h4_atr,
        )

        directional_blocks = filter_order_blocks_by_direction(
            order_blocks,
            direction,
        )

    except (ValueError, TypeError, KeyError, IndexError) as exc:
        logger.exception(
            "Erreur de calcul technique pour %s : %s",
            symbol,
            exc,
        )
        return build_analysis_error(
            symbol,
            "Impossible de calculer les niveaux techniques.",
            [str(exc)],
        )

    if not directional_blocks:
        return build_no_signal_result(
            symbol,
            "Aucun Order Block exploitable dans la direction retenue.",
            current_price,
            {
                "direction": direction,
                "supports": supports,
                "resistances": resistances,
            },
        )

    selected_block = select_nearest_order_block(
        directional_blocks,
        current_price,
    )

    if not selected_block:
        return build_no_signal_result(
            symbol,
            "Aucun Order Block n'a pu être sélectionné.",
            current_price,
        )

    block_summary = summarize_order_block(selected_block)
```

```
    try:
        confluence = calculate_confluence_score(
            current_price=current_price,
            level=levels.get("entry_level"),
            trendline=trendline,
            order_block=selected_block,
            atr=h4_atr,
            direction=direction,
        )
    except TypeError:
        # Une signature différente ne doit pas créer de faux signal.
        return build_analysis_error(
            symbol,
            "La signature de calculate_confluence_score "
            "ne correspond pas à l'appel principal.",
        )

    if not isinstance(confluence, dict):
        return build_analysis_error(
            symbol,
            "Le calcul de confluence a renvoyé un résultat invalide.",
        )

    return {
        "symbol": symbol,
        "symbol_name": get_symbol_name(symbol),
        "status": "ANALYZED",
        "signal": None,
        "direction": direction,
        "current_price": current_price,
        "timeframe_consistency": consistency,
        "d1": d1_analysis,
        "h4": h4_analysis,
        "h2": h2_analysis,
        "h1": h1_analysis,
        "levels": levels,
        "trendline": trendline,
        "order_block": block_summary,
        "confluence": confluence,
        "atr_h4": h4_atr,
        "supports": supports,
        "resistances": resistances,
        "reason": (
            "Analyse technique terminée. "
            "La validation finale de l'entrée reste à effectuer."
        ),
    }


def analyze_market(
    market_data: Dict[str, Dict[str, Any]],
) -> List[Dict[str, Any]]:
    """
    Analyse les instantanés de plusieurs indices.

    market_data doit être de la forme :
    {
        "1HZ50V": {
```

```
            "current_price": 123.45,
            "timeframes": {
                "D1": [...],
                "H4": [...],
                "H2": [...],
                "H1": [...]
            }
        }
    }
    """
    if not isinstance(market_data, dict):
        logger.error("Les données de marché doivent être un dictionnaire.")
        return []

    results = []

    for symbol, snapshot in market_data.items():
        try:
            result = analyze_symbol(symbol, snapshot)
            results.append(result)

            logger.info(
                "%s | Statut=%s | Direction=%s",
                symbol,
                result.get("status"),
                result.get("direction"),
            )

        except Exception as exc:
            logger.exception(
                "Erreur inattendue sur %s : %s",
                symbol,
                exc,
            )
            results.append(
                build_analysis_error(
                    str(symbol),
                    "Erreur inattendue pendant l'analyse.",
                    [str(exc)],
                )
            )

    return results
def extract_numeric_value(
    data: Any,
    keys: Tuple[str, ...],
) -> Optional[float]:
    """Extrait le premier nombre valide parmi plusieurs clés."""
    if not isinstance(data, dict):
        return None

    for key in keys:
        value = safe_float(data.get(key))

        if value is not None:
            return value

    return None
```

```
def extract_level_price(
    level: Any,
) -> Optional[float]:
    """Extrait le prix d'un support ou d'une résistance."""
    return extract_numeric_value(
        level,
        ("price", "level", "value", "midpoint", "middle"),
    )


def extract_zone_bounds(
    zone: Any,
) -> Optional[Tuple[float, float]]:
    """Extrait les limites d'une zone technique."""
    if not isinstance(zone, dict):
        return None

    lower = extract_numeric_value(
        zone,
        ("lower", "low", "zone_low", "bottom", "low_price"),
    )

    upper = extract_numeric_value(
        zone,
        ("upper", "high", "zone_high", "top", "high_price"),
    )

    if lower is None or upper is None:
        return None

    return min(lower, upper), max(lower, upper)


def is_valid_order_block(
    order_block: Any,
    direction: str,
) -> bool:
    """Vérifie les informations essentielles d'un Order Block."""
    if not isinstance(order_block, dict):
        return False

    block_direction = normalize_signal_direction(
        order_block.get(
            "direction",
            order_block.get("type", ""),
        )
    )

    if block_direction != normalize_signal_direction(direction):
        return False

    bounds = extract_zone_bounds(order_block)

    if bounds is None:
        return False
```

```
    lower, upper = bounds

    return (
        lower > 0
        and upper > 0
        and upper >= lower
    )


def is_valid_trendline(
    trendline: Any,
    minimum_touches: int = MIN_TRENDLINE_TOUCHES,
) -> bool:
    """Vérifie qu'une trendline dispose d'assez de contacts."""
    if not isinstance(trendline, dict):
        return False

    touches = safe_float(
        trendline.get(
            "touches",
            trendline.get("touch_count", 0),
        ),
        0.0,
    )

    slope = safe_float(
        trendline.get(
            "slope",
            trendline.get("m"),
        )
    )

    intercept = safe_float(
        trendline.get(
            "intercept",
            trendline.get("b"),
        )
    )

    if touches is None or touches < minimum_touches:
        return False

    return slope is not None and intercept is not None


def calculate_signal_quality(
    confluence: Dict[str, Any],
    direction: str,
    order_block: Dict[str, Any],
    trendline: Optional[Dict[str, Any]],
) -> Dict[str, Any]:
    """
    Évalue la qualité d'une configuration sans garantir son résultat.
    Une configuration ne devient pas automatiquement un ordre.
    """
    normalized_direction = normalize_signal_direction(direction)

    score = safe_float(
```

```
        confluence.get(
            "score",
            confluence.get("confluence_score", 0),
        ),
        0.0,
    ) or 0.0

    checks = {
        "direction_valid": normalized_direction in ("BUY", "SELL"),
        "order_block_valid": is_valid_order_block(
            order_block,
            normalized_direction,
        ),
        "trendline_valid": is_valid_trendline(trendline),
        "confluence_sufficient": score >= 3,
    }

    passed = sum(1 for value in checks.values() if value)

    return {
        "score": score,
        "checks": checks,
        "checks_passed": passed,
        "checks_total": len(checks),
        "eligible": all(checks.values()),
    }


def validate_signal_prices(
    direction: str,
    entry: float,
    stop_loss: float,
    take_profit: float,
) -> Tuple[bool, str]:
    """Vérifie que les prix sont cohérents avec le sens du signal."""
    direction = normalize_signal_direction(direction)

    if not all(
        _finite_number(value) and float(value) > 0
        for value in (entry, stop_loss, take_profit)
    ):
        return False, "Un prix est invalide ou inférieur ou égal à zéro."

    entry = float(entry)
    stop_loss = float(stop_loss)
    take_profit = float(take_profit)

    if direction == "BUY":
        if not stop_loss < entry < take_profit:
            return False, (
                "Pour un BUY, le SL doit être inférieur à l'entrée "
                "et le TP supérieur à l'entrée."
            )

    elif direction == "SELL":
        if not take_profit < entry < stop_loss:
            return False, (
                "Pour un SELL, le TP doit être inférieur à l'entrée "
```

```
                "et le SL supérieur à l'entrée."
            )

    else:
        return False, "La direction doit être BUY ou SELL."

    return True, "Prix cohérents."


def build_signal_payload(
    symbol: str,
    direction: str,
    entry: float,
    stop_loss: float,
    take_profit: float,
    current_price: float,
    quality: Dict[str, Any],
    timeframe_consistency: Dict[str, Any],
) -> Dict[str, Any]:
    """
    Construit un objet de signal pour les alertes.
    Cette fonction ne passe aucun ordre.
    """
    valid, reason = validate_signal_prices(
        direction,
        entry,
        stop_loss,
        take_profit,
    )

    if not valid:
        return build_no_signal_result(
            symbol,
            reason,
            current_price,
        )

    risk = abs(float(entry) - float(stop_loss))
    reward = abs(float(take_profit) - float(entry))

    if risk <= 0:
        return build_no_signal_result(
            symbol,
            "Risque nul : signal rejeté.",
            current_price,
        )

    risk_reward = reward / risk

    if risk_reward < MIN_RISK_REWARD:
        return build_no_signal_result(
            symbol,
            (
                f"Ratio risque/rendement insuffisant : "
                f"{risk_reward:.2f}."
            ),
            current_price,
        )
```

```
    return {
        "symbol": symbol,
        "symbol_name": get_symbol_name(symbol),
        "status": "SIGNAL_CANDIDATE",
        "signal": normalize_signal_direction(direction),
        "direction": normalize_signal_direction(direction),
        "entry": float(entry),
        "stop_loss": float(stop_loss),
        "take_profit": float(take_profit),
        "current_price": float(current_price),
        "risk_reward": round(risk_reward, 2),
        "quality": quality,
        "timeframe_consistency": timeframe_consistency,
        "automatic_execution": False,
        "reason": (
            "Configuration candidate validée techniquement. "
            "Exécution automatique désactivée."
        ),
          }
def evaluate_entry_candidate(
    symbol: str,
    direction: str,
    current_price: float,
    candles: List[Dict[str, Any]],
    supports: List[Dict[str, Any]],
    resistances: List[Dict[str, Any]],
    trendlines: Dict[str, Any],
    order_blocks: List[Dict[str, Any]],
    atr: float,
    timeframe_consistency: Dict[str, Any],
) -> Dict[str, Any]:
    """
    Évalue une entrée anticipée à partir d'une confluence technique.

    Cette fonction :
    - privilégie les supports et résistances proches ;
    - recherche une trendline confirmée par au moins trois contacts ;
    - vérifie la proximité d'un Order Block ;
    - calcule une entrée, un SL et un TP ;
    - rejette les configurations incomplètes.

    Elle ne passe aucun ordre.
    """
    direction = normalize_signal_direction(direction)

    if direction not in ("BUY", "SELL"):
        return build_no_signal_result(
            symbol,
            "Direction non exploitable.",
            current_price,
        )

    if not _finite_number(current_price) or current_price <= 0:
        return build_no_signal_result(
            symbol,
            "Prix actuel invalide.",
        )
```

```
if not _finite_number(atr) or atr <= 0:
    return build_no_signal_result(
        symbol,
        "ATR invalide.",
        current_price,
    )

try:
    validate_candle_series(
        candles,
        minimum_count=MIN_CANDLE_COUNT,
    )
except (ValueError, TypeError, KeyError) as exc:
    return build_no_signal_result(
        symbol,
        "Historique insuffisant ou invalide.",
        current_price,
        {"error": str(exc)},
    )

if not isinstance(timeframe_consistency, dict):
    return build_no_signal_result(
        symbol,
        "Vérification multi-timeframe absente.",
        current_price,
    )

if not timeframe_consistency.get("aligned", False):
    return build_no_signal_result(
        symbol,
        "Les unités de temps ne sont pas alignées.",
        current_price,
    )

selected_level = find_nearest_level(
    current_price,
    supports if direction == "BUY" else resistances,
)

if not isinstance(selected_level, dict):
    return build_no_signal_result(
        symbol,
        "Aucun niveau technique pertinent trouvé.",
        current_price,
    )

level_price = extract_level_price(selected_level)

if level_price is None or level_price <= 0:
    return build_no_signal_result(
        symbol,
        "Le niveau technique sélectionné est invalide.",
        current_price,
    )

maximum_distance = float(atr) * MAX_PRICE_DISTANCE_ATR
```

```
    if abs(float(current_price) - level_price) > maximum_distance:
        return build_no_signal_result(
            symbol,
            "Le prix est encore trop éloigné du niveau technique.",
            current_price,
            {
                "level_price": level_price,
                "distance": abs(float(current_price) - level_price),
                "maximum_distance": maximum_distance,
            },
        )

    if not isinstance(trendlines, dict):
        trendlines = {}

    trendline_key = "uptrend" if direction == "BUY" else "downtrend"
    trendline = trendlines.get(trendline_key)

    if not is_valid_trendline(trendline):
        return build_no_signal_result(
            symbol,
            "Aucune trendline valide avec trois contacts minimum.",
            current_price,
            {"selected_level": selected_level},
        )

    if not isinstance(order_blocks, list):
        order_blocks = []

    directional_blocks = filter_order_blocks_by_direction(
        order_blocks,
        direction,
    )

    if not directional_blocks:
        return build_no_signal_result(
            symbol,
            "Aucun Order Block correspondant à la direction.",
            current_price,
        )

    order_block = select_nearest_order_block(
        directional_blocks,
        current_price,
    )

    if not is_valid_order_block(order_block, direction):
        return build_no_signal_result(
            symbol,
            "Order Block sélectionné invalide.",
            current_price,
        )

    block_bounds = extract_zone_bounds(order_block)

    if block_bounds is None:
        return build_no_signal_result(
            symbol,
```

```
        "Limites de l'Order Block indisponibles.",
        current_price,
    )

block_low, block_high = block_bounds
zone_tolerance = calculate_zone_width(float(atr))

if distance_to_range(
    float(current_price),
    block_low,
    block_high,
) > zone_tolerance:
    return build_no_signal_result(
        symbol,
        "Le prix n'approche pas suffisamment l'Order Block.",
        current_price,
        {
            "order_block_low": block_low,
            "order_block_high": block_high,
            "zone_tolerance": zone_tolerance,
        },
    )

try:
    confluence = calculate_confluence_score(
        current_price=current_price,
        level=selected_level,
        trendline=trendline,
        order_block=order_block,
        atr=float(atr),
        direction=direction,
    )
except (TypeError, ValueError, KeyError) as exc:
    return build_no_signal_result(
        symbol,
        "Impossible de calculer la confluence.",
        current_price,
        {"error": str(exc)},
    )

if not isinstance(confluence, dict):
    return build_no_signal_result(
        symbol,
        "Résultat de confluence invalide.",
        current_price,
    )

quality = calculate_signal_quality(
    confluence=confluence,
    direction=direction,
    order_block=order_block,
    trendline=trendline,
)

if not quality.get("eligible", False):
    return build_no_signal_result(
        symbol,
        "La qualité de la configuration est insuffisante.",
```

```
            current_price,
            {
                "quality": quality,
                "confluence": confluence,
            },
        )

    entry = calculate_entry_price(
        direction=direction,
        current_price=float(current_price),
        zone={
            "lower": block_low,
            "upper": block_high,
        },
    )

    if not _finite_number(entry) or entry <= 0:
        return build_no_signal_result(
            symbol,
            "Prix d'entrée invalide.",
            current_price,
        )

    stop_loss = calculate_stop_loss(
        direction=direction,
        entry=float(entry),
        zone={
            "lower": block_low,
            "upper": block_high,
        },
        atr=float(atr),
    )

    if not _finite_number(stop_loss) or stop_loss <= 0:
        return build_no_signal_result(
            symbol,
            "Stop-loss invalide.",
            current_price,
        )

    target_levels = (
        resistances if direction == "BUY" else supports
    )

    valid_targets = []

    for target in target_levels:
        target_price = extract_level_price(target)

        if target_price is None:
            continue

        if direction == "BUY" and target_price > float(entry):
            valid_targets.append(target_price)

        elif direction == "SELL" and target_price < float(entry):
            valid_targets.append(target_price)
```

```
    if not valid_targets:
        return build_no_signal_result(
            symbol,
            "Aucun objectif technique disponible du bon côté du marché.",
            current_price,
        )

    if direction == "BUY":
        take_profit = min(valid_targets)
    else:
        take_profit = max(valid_targets)

    valid_prices, price_reason = validate_signal_prices(
        direction,
        float(entry),
        float(stop_loss),
        float(take_profit),
    )

    if not valid_prices:
        return build_no_signal_result(
            symbol,
            price_reason,
            current_price,
        )

    risk_reward = calculate_risk_reward(
        float(entry),
        float(stop_loss),
        float(take_profit),
    )

    if not _finite_number(risk_reward):
        return build_no_signal_result(
            symbol,
            "Ratio risque/rendement impossible à calculer.",
            current_price,
        )

    if float(risk_reward) < MIN_RISK_REWARD:
        return build_no_signal_result(
            symbol,
            "Ratio risque/rendement inférieur au minimum autorisé.",
            current_price,
            {"risk_reward": float(risk_reward)},
        )

    signal = build_signal_payload(
        symbol=symbol,
        direction=direction,
        entry=float(entry),
        stop_loss=float(stop_loss),
        take_profit=float(take_profit),
        current_price=float(current_price),
        quality=quality,
        timeframe_consistency=timeframe_consistency,
    )
```

```
    if signal.get("status") != "SIGNAL_CANDIDATE":
        return signal

    signal.update({
        "status": "SIGNAL_CANDIDATE",
        "selected_level": selected_level,
        "trendline": trendline,
        "order_block": summarize_order_block(order_block),
        "confluence": confluence,
        "atr": float(atr),
        "automatic_execution": False,
    })

    return signal
def finalize_symbol_analysis(
    symbol: str,
    snapshot: Dict[str, Any],
    analysis: Dict[str, Any],
) -> Dict[str, Any]:
    """
    Effectue la validation finale d'une analyse technique.

    Contrôles :
    - cohérence des unités D1, H4, H2 et H1 ;
    - proximité d'un support ou d'une résistance ;
    - trendline avec au moins trois contacts ;
    - présence d'un Order Block exploitable ;
    - validation de l'entrée, du SL, du TP et du ratio RR.

    Aucun ordre n'est exécuté.
    """
    if not isinstance(analysis, dict):
        return build_analysis_error(
            symbol,
            "Résultat d'analyse invalide.",
        )

    if analysis.get("status") != "ANALYZED":
        return analysis

    snapshot_valid, errors = validate_market_snapshot(snapshot)

    if not snapshot_valid:
        return build_analysis_error(
            symbol,
            "Instantané de marché invalide lors de la validation finale.",
            errors,
        )

    current_price = safe_float(
        snapshot.get("current_price")
    )

    direction = normalize_signal_direction(
        analysis.get("direction")
    )

    if current_price is None or current_price <= 0:
```

```
        return build_analysis_error(
            symbol,
            "Prix actuel indisponible lors de la validation finale.",
        )

    if direction not in ("BUY", "SELL"):
        return build_no_signal_result(
            symbol,
            "Aucune direction exploitable.",
            current_price,
        )

    timeframe_consistency = analysis.get(
        "timeframe_consistency",
        {},
    )

    if not isinstance(timeframe_consistency, dict):
        return build_no_signal_result(
            symbol,
            "Vérification multi-timeframe invalide.",
            current_price,
        )

    if not timeframe_consistency.get("aligned", False):
        return build_no_signal_result(
            symbol,
            "Les unités de temps ne sont pas suffisamment alignées.",
            current_price,
        )

    h4_analysis = analysis.get("h4", {})

    if not isinstance(h4_analysis, dict):
        return build_no_signal_result(
            symbol,
            "Analyse H4 indisponible.",
            current_price,
        )

    h4_candles = snapshot["timeframes"]["H4"]

    atr = safe_float(
        h4_analysis.get("atr")
    )

    if atr is None or atr <= 0:
        atr = calculate_atr(
            h4_candles,
            period=ATR_PERIOD,
        )

    if atr is None or atr <= 0:
        return build_no_signal_result(
            symbol,
            "ATR H4 impossible à calculer.",
            current_price,
        )
```

```
supports = h4_analysis.get("supports", [])
resistances = h4_analysis.get("resistances", [])
trendlines = h4_analysis.get("trendlines", {})

if not isinstance(supports, list):
    supports = []

if not isinstance(resistances, list):
    resistances = []

if not isinstance(trendlines, dict):
    trendlines = {}

try:
    order_blocks = detect_order_blocks(
        h4_candles,
        lookback=40,
        search_back=5,
        atr=atr,
    )

    result = evaluate_entry_candidate(
        symbol=symbol,
        direction=direction,
        current_price=current_price,
        candles=h4_candles,
        supports=supports,
        resistances=resistances,
        trendlines=trendlines,
        order_blocks=order_blocks,
        atr=atr,
        timeframe_consistency=timeframe_consistency,
    )

except (ValueError, TypeError, KeyError, IndexError) as exc:
    logger.exception(
        "Validation finale impossible pour %s : %s",
        symbol,
        exc,
    )

    return build_analysis_error(
        symbol,
        "Erreur pendant la validation finale du signal.",
        [str(exc)],
    )

if not isinstance(result, dict):
    return build_analysis_error(
        symbol,
        "La validation finale a renvoyé un résultat invalide.",
    )

result["d1_direction"] = analysis.get(
    "d1", {}
).get("direction", "NEUTRAL")
```

```
    result["h4_structure"] = h4_analysis.get(
        "structure", {}
    )

    result["h2_confirmation"] = analysis.get(
        "h2", {}
    )

    result["h1_entry_context"] = analysis.get(
        "h1", {}
    )

    result["automatic_execution"] = False

    return result


def analyze_symbol_completely_strict_legacy(
    symbol: str,
    snapshot: Dict[str, Any],
) -> Dict[str, Any]:
    """
    Point d'entrée complet pour analyser un indice et rechercher
    un signal potentiel.

    Cette fonction peut être appelée par scheduled_analysis.py.
    """
    try:
        preliminary = analyze_symbol(
            symbol=symbol,
            snapshot=snapshot,
        )

        if preliminary.get("status") != "ANALYZED":
            return preliminary

        return finalize_symbol_analysis(
            symbol=symbol,
            snapshot=snapshot,
            analysis=preliminary,
        )

    except Exception as exc:
        logger.exception(
            "Erreur complète d'analyse sur %s : %s",
            symbol,
            exc,
        )

        return build_analysis_error(
            symbol,
            "Une erreur inattendue a interrompu l'analyse.",
            [str(exc)],
        )


def analyze_multiple_symbols(
    market_data: Dict[str, Dict[str, Any]],
```

```
) -> List[Dict[str, Any]]:
    """
    Analyse plusieurs indices et renvoie un résultat par indice.
    Les erreurs sur un indice n'interrompent pas les autres analyses.
    """
    if not isinstance(market_data, dict):
        logger.error(
            "Impossible d'analyser les marchés : données invalides."
        )
        return []

    results = []

    for symbol, snapshot in market_data.items():
        result = analyze_symbol_completely(
            symbol=str(symbol),
            snapshot=snapshot,
        )

        results.append(result)

        logger.info(
            "%s | %s | Direction : %s",
            result.get("symbol_name", symbol),
            result.get("status", "UNKNOWN"),
            result.get("direction", "NEUTRAL"),
        )

    return results


def get_valid_signal_candidates(
    analysis_results: List[Dict[str, Any]],
) -> List[Dict[str, Any]]:
    """
    Filtre les résultats pour ne conserver que les candidats
    disposant d'une entrée, d'un SL et d'un TP valides.

    Un candidat reste une alerte à examiner manuellement.
    """
    if not isinstance(analysis_results, list):
        return []

    candidates = []

    for result in analysis_results:
        if not isinstance(result, dict):
            continue

        if result.get("status") != "SIGNAL_CANDIDATE":
            continue

        if result.get("automatic_execution") is not False:
            continue

        direction = normalize_signal_direction(
            result.get("direction")
        )
```

```
        entry = safe_float(result.get("entry"))
        stop_loss = safe_float(result.get("stop_loss"))
        take_profit = safe_float(result.get("take_profit"))

        if direction not in ("BUY", "SELL"):
            continue

        if entry is None or stop_loss is None or take_profit is None:
            continue

        prices_valid, _ = validate_signal_prices(
            direction,
            entry,
            stop_loss,
            take_profit,
        )

        if not prices_valid:
            continue

        risk_reward = calculate_risk_reward(
            entry,
            stop_loss,
            take_profit,
        )

        if risk_reward is None:
            continue

        if risk_reward < MIN_RISK_REWARD:
            continue

        candidates.append(result)

    return candidates
def format_price(value: Any, decimals: int = 5) -> str:
    """Formate un prix pour les journaux et les alertes."""
    number = safe_float(value)

    if number is None:
        return "N/D"

    if not isinstance(decimals, int) or decimals < 0:
        decimals = 5

    return f"{number:.{decimals}f}"


def format_risk_reward(value: Any) -> str:
    """Formate le ratio risque/rendement."""
    number = safe_float(value)

    if number is None or number < 0:
        return "N/D"

    return f"1:{number:.2f}"
```

```
def get_signal_summary(result: Dict[str, Any]) -> Dict[str, Any]:
    """Extrait les informations essentielles d'un résultat."""
    if not isinstance(result, dict):
        return {
            "status": "ERROR",
            "signal": None,
            "reason": "Résultat invalide.",
        }

    return {
        "symbol": result.get("symbol", "UNKNOWN"),
        "symbol_name": result.get("symbol_name", "Unknown index"),
        "status": result.get("status", "UNKNOWN"),
        "direction": result.get("direction", "NEUTRAL"),
        "current_price": safe_float(result.get("current_price")),
        "entry": safe_float(result.get("entry")),
        "stop_loss": safe_float(result.get("stop_loss")),
        "take_profit": safe_float(result.get("take_profit")),
        "risk_reward": safe_float(result.get("risk_reward")),
        "reason": result.get("reason", ""),
        "automatic_execution": False,
    }


def format_signal_message(result: Dict[str, Any]) -> str:
    """
    Prépare le texte d'une alerte de signal potentiel.
    Cette fonction ne communique pas avec Telegram et n'exécute
    aucun ordre.
    """
    summary = get_signal_summary(result)

    if summary.get("status") != "SIGNAL_CANDIDATE":
        return (
            f"ANALYSE — {summary.get('symbol_name', 'Indice inconnu')}\n"
            f"Statut : {summary.get('status', 'UNKNOWN')}\n"
            f"Motif : {summary.get('reason', 'Aucun détail disponible')}"
        )

    direction = summary.get("direction", "NEUTRAL")
    direction_label = "ACHAT (BUY)" if direction == "BUY" else "VENTE (SELL)"

    entry = summary.get("entry")
    stop_loss = summary.get("stop_loss")
    take_profit = summary.get("take_profit")
    current_price = summary.get("current_price")
    risk_reward = summary.get("risk_reward")

    return (
        "TRADER AI PRO — SIGNAL POTENTIEL\n"
        "━━━━━━━━━━━━━━━━━━━━\n"
        f"Indice : {summary.get('symbol_name')}\n"
        f"Symbole : {summary.get('symbol')}\n"
        f"Direction : {direction_label}\n"
        f"Prix observé : {format_price(current_price)}\n"
        "━━━━━━━━━━━━━━━━━━━━\n"
        f"Entrée indicative : {format_price(entry)}\n"
```

```
        f"Stop-loss (SL) : {format_price(stop_loss)}\n"
        f"Take-profit (TP) : {format_price(take_profit)}\n"
        f"Ratio risque/rendement : {format_risk_reward(risk_reward)}\n"
        "━━━━━━━━━━━━━━━━━━━━\n"
        "Analyse : D1 / H4 / H2 / H1\n"
        "Exécution automatique : DÉSACTIVÉE\n"
        "Mode : ALERTE UNIQUEMENT\n"
        "Vérifier le graphique avant toute décision."
    )


def log_analysis_result(result: Dict[str, Any]) -> None:
    """Écrit un résumé exploitable dans les journaux."""
    if not isinstance(result, dict):
        logger.error("Impossible de journaliser un résultat invalide.")
        return

    symbol_name = result.get("symbol_name", "Indice inconnu")
    symbol = result.get("symbol", "UNKNOWN")
    status = result.get("status", "UNKNOWN")
    direction = result.get("direction", "NEUTRAL")

    if status == "SIGNAL_CANDIDATE":
        logger.info(
            "%s (%s) | CANDIDAT %s | Entrée=%s | SL=%s | TP=%s | RR=%s",
            symbol_name,
            symbol,
            direction,
            format_price(result.get("entry")),
            format_price(result.get("stop_loss")),
            format_price(result.get("take_profit")),
            format_risk_reward(result.get("risk_reward")),
        )
    else:
        logger.info(
            "%s (%s) | Statut=%s | Direction=%s | Motif=%s",
            symbol_name,
            symbol,
            status,
            direction,
            result.get("reason", "Non précisé"),
        )


def log_multiple_analysis_results(
    results: List[Dict[str, Any]],
) -> Dict[str, int]:
    """Journalise un ensemble de résultats et compte les statuts."""
    counts = {
        "total": 0,
        "signal_candidates": 0,
        "no_signal": 0,
        "errors": 0,
        "other": 0,
    }

    if not isinstance(results, list):
        logger.error("La liste des résultats est invalide.")
```

```
        return counts

    for result in results:
        if not isinstance(result, dict):
            counts["errors"] += 1
            continue

        counts["total"] += 1
        status = result.get("status", "UNKNOWN")

        if status == "SIGNAL_CANDIDATE":
            counts["signal_candidates"] += 1
        elif status == "NO_SIGNAL":
            counts["no_signal"] += 1
        elif status == "ERROR":
            counts["errors"] += 1
        else:
            counts["other"] += 1

        log_analysis_result(result)

    logger.info(
        "BILAN | Total=%d | Candidats=%d | Sans signal=%d | Erreurs=%d",
        counts["total"],
        counts["signal_candidates"],
        counts["no_signal"],
        counts["errors"],
    )

    return counts
def get_strategy_config() -> Dict[str, Any]:
    """Retourne la configuration active de la stratégie."""
    return {
        "name": "TRADER AI PRO",
        "version": "2.0",
        "strategy": "Support/Resistance + Trendlines + Order Blocks",
        "timeframes": {
            "D1": {
                "required_candles": TIMEFRAME_REQUIREMENTS["D1"],
                "role": "Direction principale",
            },
            "H4": {
                "required_candles": TIMEFRAME_REQUIREMENTS["H4"],
                "role": "Structure et zones techniques",
            },
            "H2": {
                "required_candles": TIMEFRAME_REQUIREMENTS["H2"],
                "role": "Filtre de confirmation",
            },
            "H1": {
                "required_candles": TIMEFRAME_REQUIREMENTS["H1"],
                "role": "Précision de l'entrée",
            },
        },
        "minimum_trendline_touches": MIN_TRENDLINE_TOUCHES,
        "minimum_risk_reward": MIN_RISK_REWARD,
        "atr_period": ATR_PERIOD,
        "atr_stop_buffer": ATR_STOP_BUFFER,
```

```
        "zone_atr_tolerance": ZONE_ATR_TOLERANCE,
        "max_price_distance_atr": MAX_PRICE_DISTANCE_ATR,
        "minimum_candles": MIN_CANDLE_COUNT,
        "automatic_execution": False,
    }


def validate_strategy_configuration() -> Dict[str, Any]:
    """
    Vérifie les paramètres de configuration sans contacter Deriv
    et sans produire de signal de trading.
    """
    errors = []
    config = get_strategy_config()

    if not isinstance(MIN_TRENDLINE_TOUCHES, int):
        errors.append(
            "MIN_TRENDLINE_TOUCHES doit être un entier."
        )
    elif MIN_TRENDLINE_TOUCHES < 3:
        errors.append(
            "Une trendline doit exiger au moins trois contacts."
        )

    if not _finite_number(MIN_RISK_REWARD):
        errors.append(
            "MIN_RISK_REWARD doit être un nombre valide."
        )
    elif MIN_RISK_REWARD < 1.0:
        errors.append(
            "Le ratio risque/rendement minimum ne peut pas être inférieur à 1."
        )

    if not isinstance(TIMEFRAME_REQUIREMENTS, dict):
        errors.append(
            "TIMEFRAME_REQUIREMENTS doit être un dictionnaire."
        )
    else:
        required_timeframes = ("D1", "H4", "H2", "H1")

        for timeframe in required_timeframes:
            count = TIMEFRAME_REQUIREMENTS.get(timeframe)

            if not isinstance(count, int) or count < 200:
                errors.append(
                    f"{timeframe} doit exiger au moins 200 bougies."
                )

    if not _finite_number(ATR_PERIOD):
        errors.append("ATR_PERIOD est invalide.")
    elif ATR_PERIOD <= 0:
        errors.append("ATR_PERIOD doit être positif.")

    if not _finite_number(ATR_STOP_BUFFER):
        errors.append("ATR_STOP_BUFFER est invalide.")
    elif ATR_STOP_BUFFER < 0:
        errors.append("ATR_STOP_BUFFER ne peut pas être négatif.")
```

```
    if not _finite_number(ZONE_ATR_TOLERANCE):
        errors.append("ZONE_ATR_TOLERANCE est invalide.")
    elif ZONE_ATR_TOLERANCE <= 0:
        errors.append("ZONE_ATR_TOLERANCE doit être positif.")

    if config.get("automatic_execution") is not False:
        errors.append(
            "L'exécution automatique doit rester désactivée."
        )

    return {
        "valid": len(errors) == 0,
        "errors": errors,
        "configuration": config,
    }


def validate_supported_symbols(
    symbols: List[str],
) -> Dict[str, Any]:
    """Vérifie les symboles à analyser."""
    if not isinstance(symbols, list):
        return {
            "valid": False,
            "valid_symbols": [],
            "invalid_symbols": [],
            "error": "La liste des symboles est invalide.",
        }

    valid_symbols = []
    invalid_symbols = []

    for symbol in symbols:
        if not isinstance(symbol, str):
            invalid_symbols.append(str(symbol))
            continue

        normalized = symbol.strip()

        if normalized in SYMBOL_NAMES:
            valid_symbols.append(normalized)
        else:
            invalid_symbols.append(normalized)

    return {
        "valid": len(invalid_symbols) == 0,
        "valid_symbols": valid_symbols,
        "invalid_symbols": invalid_symbols,
        "total": len(symbols),
    }


def run_strategy_diagnostics() -> Dict[str, Any]:
    """
    Lance uniquement les diagnostics locaux.
    Aucun accès réseau, aucune alerte Telegram et aucun ordre.
    """
    logger.info(
```

```
        "Démarrage des diagnostics de Trader AI Pro."
    )

    configuration_check = validate_strategy_configuration()

    symbols_check = validate_supported_symbols(
        list(SYMBOL_NAMES.keys())
    )

    passed = (
        configuration_check["valid"]
        and symbols_check["valid"]
    )

    report = {
        "status": "PASS" if passed else "FAIL",
        "configuration": configuration_check,
        "symbols": symbols_check,
        "automatic_execution": False,
    }

    if passed:
        logger.info(
            "Diagnostics locaux réussis."
        )
    else:
        logger.error(
            "Diagnostics locaux échoués : configuration à corriger."
        )

        for error in configuration_check.get("errors", []):
            logger.error("Configuration : %s", error)

    return report
def run_strategy_self_tests() -> Dict[str, Any]:
    """
    Exécute des tests locaux de fonctions élémentaires.
    Ces tests ne contactent aucun service externe et ne génèrent
    aucun signal de trading.
    """
    tests = []

    def record_test(
        name: str,
        passed: bool,
        details: str = "",
    ) -> None:
        tests.append({
            "name": name,
            "passed": bool(passed),
            "details": details,
        })

    # Test 1 : reconnaissance des symboles.
    try:
        record_test(
            "symbol_name",
            get_symbol_name("1HZ50V") != "1HZ50V",
```

```
        "Vérification du nom complet d'un indice.",
    )
except Exception as exc:
    record_test("symbol_name", False, str(exc))

# Test 2 : validation des prix d'un BUY.
try:
    valid, _ = validate_signal_prices(
        "BUY",
        entry=100.0,
        stop_loss=95.0,
        take_profit=110.0,
    )
    record_test(
        "valid_buy_prices",
        valid,
        "SL < entrée < TP.",
    )
except Exception as exc:
    record_test("valid_buy_prices", False, str(exc))

# Test 3 : validation des prix d'un SELL.
try:
    valid, _ = validate_signal_prices(
        "SELL",
        entry=100.0,
        stop_loss=105.0,
        take_profit=90.0,
    )
    record_test(
        "valid_sell_prices",
        valid,
        "TP < entrée < SL.",
    )
except Exception as exc:
    record_test("valid_sell_prices", False, str(exc))

# Test 4 : rejet d'un BUY dont les prix sont incohérents.
try:
    valid, _ = validate_signal_prices(
        "BUY",
        entry=100.0,
        stop_loss=105.0,
        take_profit=110.0,
    )
    record_test(
        "reject_invalid_buy",
        not valid,
        "Le stop-loss ne doit pas dépasser l'entrée.",
    )
except Exception as exc:
    record_test("reject_invalid_buy", False, str(exc))

# Test 5 : calcul de la distance à une zone.
try:
    distance = distance_to_range(
        price=105.0,
        lower_bound=100.0,
```

```
        upper_bound=110.0,
    )
    record_test(
        "price_inside_range",
        distance == 0.0,
        f"Distance calculée : {distance}.",
    )
except Exception as exc:
    record_test("price_inside_range", False, str(exc))

# Test 6 : calcul de la distance à l'extérieur d'une zone.
try:
    distance = distance_to_range(
        price=115.0,
        lower_bound=100.0,
        upper_bound=110.0,
    )
    record_test(
        "price_outside_range",
        math.isclose(distance, 5.0),
        f"Distance calculée : {distance}.",
    )
except Exception as exc:
    record_test("price_outside_range", False, str(exc))

# Test 7 : conversion numérique.
try:
    result = safe_float("123.45")
    record_test(
        "safe_float",
        result == 123.45,
        f"Valeur obtenue : {result}.",
    )
except Exception as exc:
    record_test("safe_float", False, str(exc))

# Test 8 : rejet des valeurs non finies.
try:
    result = safe_float(float("nan"))
    record_test(
        "reject_nan",
        result is None,
        f"Valeur obtenue : {result}.",
    )
except Exception as exc:
    record_test("reject_nan", False, str(exc))

# Test 9 : configuration de la stratégie.
try:
    config = validate_strategy_configuration()
    record_test(
        "strategy_configuration",
        config.get("valid") is True,
        str(config.get("errors", [])),
    )
except Exception as exc:
    record_test("strategy_configuration", False, str(exc))
```

```
    # Bilan des tests.
    passed_count = sum(
        1 for test in tests if test["passed"]
    )

    failed_count = len(tests) - passed_count

    report = {
        "status": "PASS" if failed_count == 0 else "FAIL",
        "total_tests": len(tests),
        "passed": passed_count,
        "failed": failed_count,
        "tests": tests,
        "automatic_execution": False,
    }

    for test in tests:
        if test["passed"]:
            logger.info(
                "[TEST OK] %s",
                test["name"],
            )
        else:
            logger.error(
                "[TEST ÉCHEC] %s : %s",
                test["name"],
                test["details"],
            )

    logger.info(
        "Bilan des tests : %d réussis, %d échoués, sur %d.",
        passed_count,
        failed_count,
        len(tests),
    )

    return report
def get_strategy_health_report() -> Dict[str, Any]:
    """
    Produit un rapport local sur l'état de la stratégie.
    Ne contacte aucun service externe.
    """
    logger.info("Contrôle de santé de Trader AI Pro.")

    configuration = validate_strategy_configuration()
    self_tests = run_strategy_self_tests()

    required_functions = [
        "analyze_symbol",
        "analyze_symbol_completely",
        "analyze_multiple_symbols",
        "evaluate_entry_candidate",
        "calculate_stop_loss",
        "calculate_take_profit",
        "calculate_risk_reward",
        "detect_trendlines",
        "detect_order_blocks",
        "calculate_confluence_score",
```

```
        "format_signal_message",
    ]

    missing_functions = [
        name
        for name in required_functions
        if name not in globals()
        or not callable(globals().get(name))
    ]

    passed = (
        configuration.get("valid", False)
        and self_tests.get("status") == "PASS"
        and not missing_functions
    )

    report = {
        "status": "PASS" if passed else "FAIL",
        "configuration_valid": configuration.get("valid", False),
        "self_tests": self_tests,
        "missing_functions": missing_functions,
        "automatic_execution": False,
    }

    if passed:
        logger.info(
            "Contrôle local terminé : configuration et tests validés."
        )
    else:
        logger.error(
            "Contrôle local échoué. Vérifier les erreurs avant utilisation."
        )

        if missing_functions:
            logger.error(
                "Fonctions manquantes : %s",
                ", ".join(missing_functions),
            )

    return report


def print_strategy_health_report() -> Dict[str, Any]:
    """Affiche un résumé lisible du contrôle local."""
    report = get_strategy_health_report()

    print("=" * 54)
    print("TRADER AI PRO — DIAGNOSTIC DE LA STRATÉGIE")
    print("=" * 54)
    print(f"Statut général : {report['status']}")
    print(
        "Configuration valide : "
        f"{report['configuration_valid']}"
    )

    test_report = report["self_tests"]

    print(
```

```
        "Tests locaux : "
        f"{test_report['passed']}/{test_report['total_tests']} réussis"
    )

    if report["missing_functions"]:
        print("Fonctions manquantes :")
        for function_name in report["missing_functions"]:
            print(f"- {function_name}")

    print("Exécution automatique des ordres : DÉSACTIVÉE")
    print("=" * 54)

    return report
def get_strategy_metadata() -> Dict[str, Any]:
    """Retourne les informations générales sur la stratégie."""
    return {
        "name": "Trader AI Pro",
        "version": "2.0",
        "description": (
            "Analyse multi-timeframe avec supports, résistances, "
            "trendlines et Order Blocks."
        ),
        "timeframes": ["D1", "H4", "H2", "H1"],
        "minimum_candles_per_timeframe": 200,
        "minimum_trendline_touches": MIN_TRENDLINE_TOUCHES,
        "minimum_risk_reward": MIN_RISK_REWARD,
        "automatic_execution": False,
        "signal_mode": "ALERT_ONLY",
    }


def prepare_strategy_run() -> Dict[str, Any]:
    """
    Prépare les informations nécessaires avant une analyse.
    Cette fonction ne récupère pas les données de marché.
    """
    config_check = validate_strategy_configuration()
    symbols_check = validate_supported_symbols(
        list(SYMBOL_NAMES.keys())
    )

    errors = []

    if not config_check.get("valid", False):
        errors.extend(config_check.get("errors", []))

    if not symbols_check.get("valid", False):
        errors.append(
            "La liste des symboles contient des valeurs invalides."
        )

    return {
        "ready": len(errors) == 0,
        "metadata": get_strategy_metadata(),
        "configuration": config_check,
        "symbols": symbols_check,
        "errors": errors,
        "automatic_execution": False,
```

```
    }


def get_strategy_status() -> Dict[str, Any]:
    """Retourne un statut synthétique de la stratégie."""
    preparation = prepare_strategy_run()

    return {
        "status": "READY_FOR_COMPATIBILITY_CHECK"
        if preparation["ready"]
        else "CONFIGURATION_ERROR",
        "ready": preparation["ready"],
        "timeframes": ["D1", "H4", "H2", "H1"],
        "candles_required": 200,
        "automatic_execution": False,
        "errors": preparation["errors"],
      }


# ============================================================
# STRATEGIE PAR ZONES S/R (v2) : ALERTE ANTICIPEE, PRIX ENCORE LOIN
# ============================================================
# Declencheur unique : une zone S/R majeure (H4 + D1), meme si le prix
# en est encore eloigne. Le bot envoie alors un plan "ordre limite" :
#   - zone retournee (ancien support <-> resistance) = retest ;
#   - trendline (y compris cassee puis retestee) qui croise la zone :
#     son intersection avec la zone devient le point d'entree ;
#   - Order Block / Doji (H1 puis H4) dans la zone : ils la RESSERRENT
#     (entree precise, SL proche) ; ils ne bloquent jamais l'alerte.
# Aucun ordre n'est execute : alerte uniquement.

SETUP_TYPE_ZONE = "ZONE_ANTICIPATION"

import os as _os


def _env_number(name: str, default: float, minimum: float = 0.0) -> float:
    try:
        return max(minimum, float(_os.getenv(name, str(default))))
    except ValueError:
        return default


# Reglables par variables d'environnement.
MIN_ZONE_TOUCHES = int(_env_number("MIN_ZONE_TOUCHES", 2, 1))
MAX_ALERT_DISTANCE_ATR = _env_number("MAX_ALERT_DISTANCE_ATR", 8.0, 0.5)
MIN_SIGNAL_GRADE = _os.getenv("MIN_SIGNAL_GRADE", "C").strip().upper()
GRADE_RANK = {"C": 1, "B": 2, "A": 3}

PROJECTION_CANDLES_H4 = 60      # horizon de projection des trendlines
INTERSECTION_BAND_ATR = 0.15    # demi-largeur de la zone autour de l'intersection
SCAN_CANDIDATES_PER_SIDE = 4    # zones evaluees par sens
REFINEMENT_LOOKBACK = 150       # OB recherches dans la zone
DOJI_LOOKBACK = 60              # dojis recents seulement
FLIP_LOOKBACK_H4 = 150          # recherche d'une cassure de la zone
FLIP_MIN_CLOSES = 12            # clotures H4 de l'autre cote = vraie cassure
```

```
def _zone_of(level: Dict[str, Any], atr: float) -> Optional[Tuple[float, float]]:
    try:
        lower = float(level.get("lower", level.get("price")))
        upper = float(level.get("upper", level.get("price")))
    except (TypeError, ValueError):
        return None
    if not (math.isfinite(lower) and math.isfinite(upper)):
        return None
    if lower > upper:
        lower, upper = upper, lower
    minimum_width = atr * ZONE_ATR_TOLERANCE
    if upper - lower < minimum_width:
        middle = (upper + lower) / 2.0
        lower, upper = middle - minimum_width / 2.0, middle + minimum_width / 2.0
    return lower, upper


def _overlap(a: Tuple[float, float], b: Tuple[float, float]) -> Optional[Tuple[float, float]]:
    low, high = max(a[0], b[0]), min(a[1], b[1])
    return (low, high) if low <= high else None


def collect_candidate_zones(
    price: float,
    atr: float,
    supports: List[Dict[str, Any]],
    resistances: List[Dict[str, Any]],
) -> List[Dict[str, Any]]:
    """Fusionne supports et resistances (zones retournees incluses)."""
    zones: List[Dict[str, Any]] = []
    gap = atr * ZONE_ATR_TOLERANCE
    for level in list(supports) + list(resistances):
        if not isinstance(level, dict):
            continue
        bounds = _zone_of(level, atr)
        if bounds is None:
            continue
        lower, upper = bounds
        touches = int(level.get("touches", 1) or 1)
        merged = False
        for known in zones:
            if _overlap((known["lower"] - gap, known["upper"] + gap), (lower, upper)):
                known["lower"] = min(known["lower"], lower)
                known["upper"] = max(known["upper"], upper)
                known["touches"] += touches
                merged = True
                break
        if not merged:
            zones.append({"lower": lower, "upper": upper, "touches": touches})
    for zone in zones:
        if zone["lower"] <= price <= zone["upper"]:
            zone["distance"], zone["position"] = 0.0, "INSIDE"
        elif price > zone["upper"]:
            zone["distance"], zone["position"] = price - zone["upper"], "BELOW_PRICE"
        else:
            zone["distance"], zone["position"] = zone["lower"] - price, "ABOVE_PRICE"
    return zones
```

```
def _flipped_zone(h4_candles: List[Dict[str, Any]], zone: Tuple[float, float], direction: str)
-> Optional[str]:
    """Zone retournee : le prix l'a franchie durablement puis revient la retester."""
    recent = h4_candles[-FLIP_LOOKBACK_H4:]
    if direction == "SELL":   # zone au-dessus : ancien support passe sous le prix
        if sum(1 for c in recent if c["close"] > zone[1]) >= FLIP_MIN_CLOSES:
            return "Zone retournee : ancien support devenu resistance (retest)"
    else:
        if sum(1 for c in recent if c["close"] < zone[0]) >= FLIP_MIN_CLOSES:
            return "Zone retournee : ancienne resistance devenue support (retest)"
    return None


def _trendline_intersection(
    trendlines: Dict[str, Any],
    direction: str,
    zone: Tuple[float, float],
    price: float,
    now_index: int,
    atr: float,
) -> Optional[Dict[str, Any]]:
    """Trendline (cassee ou non) qui croise la zone, aujourd'hui ou bientot."""
    tolerance = atr * 0.05
    best = None
    for kind in ("uptrend", "downtrend"):
        lines = trendlines.get(kind) if isinstance(trendlines, dict) else None
        if isinstance(lines, dict):
            lines = [lines]
        for line in lines or []:
            if not isinstance(line, dict):
                continue
            if int(line.get("touches", 0) or 0) < MIN_TRENDLINE_TOUCHES:
                continue
            slope, intercept = safe_float(line.get("slope")), safe_float(line.get("intercept"))
            if slope is None or intercept is None:
                continue
            now_value = slope * now_index + intercept
            # Une ligne sous le prix agit en support, au-dessus en resistance.
            if direction == "BUY" and now_value > price:
                continue
            if direction == "SELL" and now_value < price:
                continue
            for step in range(0, PROJECTION_CANDLES_H4 + 1):
                value = slope * (now_index + step) + intercept
                if zone[0] - tolerance <= value <= zone[1] + tolerance:
                    broken = (kind == "uptrend") == (direction == "SELL")
                    candidate = {
                        "price": min(max(value, zone[0]), zone[1]),
                        "candles_ahead": step,
                        "touches": int(line.get("touches", 0)),
                        "kind": kind,
                        "broken_retest": broken,
                        "slope": slope,
                        "intercept": intercept,
                        "start_index": line.get("start_index"),
                    }
                    if best is None or step < best["candles_ahead"]:
```

```
                        best = candidate
                    break
    return best


def _refinement_candidates(
    direction: str,
    zone: Tuple[float, float],
    h1: List[Dict[str, Any]],
    h4: List[Dict[str, Any]],
    atr_h1: float,
    atr_h4: float,
) -> List[Dict[str, Any]]:
    """Order Blocks et dojis (H1 puis H4) qui se trouvent dans la zone."""
    wanted = "BULLISH" if direction == "BUY" else "BEARISH"
    expanded = (zone[0] - atr_h4 * ZONE_ATR_TOLERANCE, zone[1] + atr_h4 * ZONE_ATR_TOLERANCE)
    found: List[Dict[str, Any]] = []
    for label, candles, atr in (("H1", h1, atr_h1), ("H4", h4, atr_h4)):
        try:
            blocks = detect_order_blocks(candles, lookback=REFINEMENT_LOOKBACK, search_back=5,
atr=atr)
        except (StrategyError, ValueError, TypeError, KeyError, IndexError):
            blocks = []
        for block in blocks:
            if str(block.get("type", "")).upper() != wanted:
                continue
            lo, hi = safe_float(block.get("zone_lower")), safe_float(block.get("zone_upper"))
            if lo is None or hi is None or lo > hi or _overlap(zone, (lo, hi)) is None:
                continue
            found.append({"label": f"OB {label}", "bounds": (lo, hi), "atr": atr, "kind": "OB",
"tf": label})
        for candle in candles[-DOJI_LOOKBACK:]:
            try:
                if not is_doji(candle, 0.10):
                    continue
            except StrategyError:
                continue
            lo, hi = float(candle["low"]), float(candle["high"])
            if _overlap(zone, (lo, hi)) is None:
                continue
            found.append({"label": f"Doji {label}", "bounds": (lo, hi), "atr": atr, "kind":
"DOJI", "tf": label})
    return found


def _build_zone_signal(
    symbol: str,
    zone: Dict[str, Any],
    direction: str,
    ctx: Dict[str, Any],
) -> Optional[Dict[str, Any]]:
    price, atr_h4, atr_h1 = ctx["price"], ctx["atr_h4"], ctx["atr_h1"]
    h1, h4, zones = ctx["h1"], ctx["h4"], ctx["zones"]
    base = (zone["lower"], zone["upper"])
    confluences: List[str] = []
    refinements: List[str] = []
    score = 0
```

```
    if zone["touches"] >= 4:
        confluences.append(f"Zone testee {zone['touches']} fois")
        score += 1

    flipped = _flipped_zone(h4, base, direction)
    if flipped:
        confluences.append(flipped)
        score += 1

    intersection = _trendline_intersection(
        ctx["trendlines"], direction, base, price, len(h4) - 1, atr_h4
    )
    if intersection:
        kind = "haussiere" if intersection["kind"] == "uptrend" else "baissiere"
        role = "cassee, retest" if intersection["broken_retest"] else "respectee"
        when = "maintenant" if intersection["candles_ahead"] == 0 else f"dans ~
{intersection['candles_ahead']} bougies H4"
        confluences.append(
            f"Trendline {kind} ({intersection['touches']} contacts, {role}) croise la zone
{when}"
        )
        score += 1

    anchor = intersection["price"] if intersection else None
    refined, atr_ref = base, atr_h4
    candidates = _refinement_candidates(direction, base, h1, h4, atr_h1, atr_h4)
    tolerance = atr_h4 * ZONE_ATR_TOLERANCE

    def rank(item):
        common = _overlap(base, item["bounds"])
        if common is None:
            return None
        near_anchor = anchor is not None and (common[0] - tolerance <= anchor <= common[1] +
tolerance)
        return (0 if near_anchor else 1, 0 if item["tf"] == "H1" else 1, common[1] -
common[0]), common

    ranked = [(rank(c), c) for c in candidates if rank(c) is not None]
    used_kinds = set()
    if ranked:
        ranked.sort(key=lambda pair: pair[0][0])
        (_, common), best = ranked[0]
        if (common[1] - common[0]) < (refined[1] - refined[0]):
            refined, atr_ref = common, best["atr"]
        for _, item in ranked:
            if item["kind"] not in used_kinds:
                used_kinds.add(item["kind"])
                refinements.append(item["label"])
                confluences.append(f"{item['label'].replace('OB', 'Order Block')} dans la
zone")
                score += 1
    if not used_kinds and anchor is not None:
        band = atr_h4 * INTERSECTION_BAND_ATR
        narrowed = _overlap(base, (anchor - band, anchor + band))
        if narrowed:
            refined = narrowed
            refinements.append("Intersection trendline x zone")
```

```
    sweep = detect_liquidity_sweep(h1)
    if sweep.get("detected") and normalize_signal_direction(sweep.get("direction")) ==
direction:
        if distance_to_range(float(h1[-1]["close"]), base[0], base[1]) <= atr_h1 *
MAX_PRICE_DISTANCE_ATR:
            confluences.append("Balayage de liquidite H1")
            score += 1

    if distance_to_range(float(h1[-1]["close"]), base[0], base[1]) <= atr_h1 *
MAX_PRICE_DISTANCE_ATR:
        if detect_rejection_candle(h1[-1], direction).get("detected"):
            confluences.append("Bougie de rejet H1")
            score += 1
        engulfing = detect_engulfing(h1)
        if engulfing.get("detected") and normalize_signal_direction(engulfing.get("direction"))
== direction:
            confluences.append("Engulfing H1")
            score += 1

    trend_d1, trend_h4 = ctx["trend_d1"], ctx["trend_h4"]
    if trend_d1 == direction or (trend_d1 == "NEUTRAL" and trend_h4 == direction):
        confluences.append("Sens conforme a la tendance D1/H4")
        score += 1
    counter_trend = trend_d1 in ("BUY", "SELL") and trend_d1 != direction

    # --- Entree / SL / TP -----------------------------------
    point = anchor if (anchor is not None and refined[0] <= anchor <= refined[1]) else (
        refined[1] if direction == "BUY" else refined[0]
    )
    buffer = atr_ref * ATR_STOP_BUFFER
    if direction == "BUY":
        entry = min(price, point)
        stop_loss = min(refined[0], entry) - buffer
    else:
        entry = max(price, point)
        stop_loss = max(refined[1], entry) + buffer
    risk = abs(entry - stop_loss)
    if risk <= 0 or stop_loss <= 0 or entry <= 0:
        return None

    targets = []
    for other in zones:
        if other is zone:
            continue
        if direction == "BUY" and other["lower"] > entry:
            targets.append(other["lower"])
        elif direction == "SELL" and other["upper"] < entry:
            targets.append(other["upper"])
    targets.sort(reverse=(direction == "SELL"))
    take_profit, tp_source = None, "NIVEAU_TECHNIQUE"
    for target in targets:
        if abs(target - entry) / risk >= MIN_RISK_REWARD:
            take_profit = target
            break
    if take_profit is None:
        tp_source = "RR_MINIMUM"
        take_profit = entry + risk * MIN_RISK_REWARD if direction == "BUY" else entry - risk *
MIN_RISK_REWARD
```

```
    ok, _ = validate_signal_prices(direction, entry, stop_loss, take_profit)
    if not ok:
        return None
    risk_reward = round(abs(take_profit - entry) / risk, 2)
    if risk_reward < MIN_RISK_REWARD:
        return None

    grade = "A" if score >= 6 else ("B" if score >= 4 else "C")
    if GRADE_RANK.get(grade, 1) < GRADE_RANK.get(MIN_SIGNAL_GRADE, 1):
        return None

    market_entry = abs(entry - price) <= atr_h1 * 0.05
    bucket = int(round(((refined[0] + refined[1]) / 2.0) / (atr_h4 * 0.5)))
    return {
        "symbol": symbol,
        "symbol_name": get_symbol_name(symbol),
        "status": "SIGNAL_CANDIDATE",
        "setup_type": SETUP_TYPE_ZONE,
        "signal": direction,
        "direction": direction,
        "entry": float(entry),
        "entry_mode": "AU_MARCHE" if market_entry else "LIMITE_SUR_ZONE",
        "stop_loss": float(stop_loss),
        "take_profit": float(take_profit),
        "tp_source": tp_source,
        "current_price": float(price),
        "distance_atr": round(abs(entry - price) / atr_h4, 2),
        "zone_distance_atr": round(zone["distance"] / atr_h4, 2),
        "atr_h4": float(atr_h4),
        "risk_reward": risk_reward,
        "rr": risk_reward,
        "grade": grade,
        "score": score,
        "zone": {"lower": base[0], "upper": base[1], "touches": zone["touches"]},
        "refined_zone": {"lower": refined[0], "upper": refined[1]},
        "refinements": refinements,
        "confluences": confluences,
        "retest": bool(flipped) or bool(intersection and intersection["broken_retest"]),
        "intersection": intersection,
        "counter_trend": counter_trend,
        "trend_d1": trend_d1,
        "trend_h4": trend_h4,
        "signal_key": f"{symbol}|{direction}|{bucket}",
        "automatic_execution": False,
        "reason": "Zone S/R detectee : plan d'entree anticipee. Alerte uniquement.",
    }


def find_zone_signals(symbol: str, snapshot: Dict[str, Any]) -> Dict[str, Any]:
    """Analyse complete ; retourne {'signals': [...], 'result': dict}."""
    try:
        valid, errors = validate_market_snapshot(snapshot)
    except StrategyError as exc:
        valid, errors = False, [str(exc)]
    if not valid:
        return {"signals": [], "result": build_analysis_error(symbol, "Instantane de marche
incomplet ou invalide.", errors)}
```

```
    price = safe_float(snapshot.get("current_price"))
    if price is None or price <= 0:
        return {"signals": [], "result": build_analysis_error(symbol, "Le prix actuel est
invalide.")}

    tf = snapshot["timeframes"]
    try:
        d1 = analyze_d1_direction(tf["D1"])
        h4_analysis = analyze_h4_structure(tf["H4"])
        h1 = validate_candle_series(tf["H1"], minimum_count=MIN_CANDLE_COUNT)
        h4 = validate_candle_series(tf["H4"], minimum_count=MIN_CANDLE_COUNT)
        atr_h4 = safe_float(h4_analysis.get("atr")) or calculate_atr(h4, period=ATR_PERIOD)
        atr_h1 = calculate_atr(h1, period=ATR_PERIOD)
        atr_d1 = calculate_atr(tf["D1"], period=ATR_PERIOD)
        supports = find_supports(h4, atr=atr_h4) + find_supports(tf["D1"], atr=atr_d1)
        resistances = find_resistances(h4, atr=atr_h4) + find_resistances(tf["D1"], atr=atr_d1)
    except (StrategyError, ValueError, TypeError, KeyError, IndexError) as exc:
        return {"signals": [], "result": build_analysis_error(symbol, "Erreur pendant l'analyse
technique.", [str(exc)])}
    if not atr_h4 or atr_h4 <= 0 or not atr_h1 or atr_h1 <= 0:
        return {"signals": [], "result": build_no_signal_result(symbol, "ATR indisponible.",
price)}

    zones = collect_candidate_zones(price, atr_h4, supports, resistances)
    limit = atr_h4 * MAX_ALERT_DISTANCE_ATR
    usable = [z for z in zones if z["distance"] <= limit and z["touches"] >= MIN_ZONE_TOUCHES]
    if not usable:
        nearest = min(zones, key=lambda z: z["distance"]) if zones else None
        return {"signals": [], "result": build_no_signal_result(
            symbol, "Aucune zone S/R exploitable dans le rayon d'alerte.", price,
            {"zones_detected": len(zones),
             "nearest_distance_atr": round(nearest["distance"] / atr_h4, 2) if nearest else
None,
             "alert_radius_atr": MAX_ALERT_DISTANCE_ATR})}

    ctx = {
        "price": price, "atr_h4": atr_h4, "atr_h1": atr_h1, "h1": h1, "h4": h4,
        "zones": zones, "trendlines": h4_analysis.get("trendlines", {}),
        "trend_d1": normalize_signal_direction(d1.get("trend", {}).get("direction")),
        "trend_h4": normalize_signal_direction(h4_analysis.get("trend", {}).get("direction")),
    }

    by_side: Dict[str, List[Dict[str, Any]]] = {"BUY": [], "SELL": []}
    for zone in sorted(usable, key=lambda z: z["distance"]):
        if zone["position"] == "BELOW_PRICE":
            direction = "BUY"
        elif zone["position"] == "ABOVE_PRICE":
            direction = "SELL"
        else:
            direction = "NEUTRAL"
            for candle in reversed(h1[-60:-1]):
                if candle["close"] < zone["lower"]:
                    direction = "SELL"
                    break
                if candle["close"] > zone["upper"]:
                    direction = "BUY"
                    break
            if direction == "NEUTRAL" and ctx["trend_d1"] in ("BUY", "SELL"):
```

```
                direction = ctx["trend_d1"]
            if direction == "NEUTRAL":
                continue
        if len(by_side[direction]) >= SCAN_CANDIDATES_PER_SIDE:
            continue
        built = _build_zone_signal(symbol, zone, direction, ctx)
        if built:
            by_side[direction].append(built)

    signals = []
    for direction in ("BUY", "SELL"):
        if by_side[direction]:
            signals.append(sorted(
                by_side[direction],
                key=lambda s: (-s["score"], -s["zone"]["touches"], s["distance_atr"]),
            )[0])
    signals.sort(key=lambda s: (-s["score"], s["distance_atr"]))

    if not signals:
        return {"signals": [], "result": build_no_signal_result(
            symbol, "Zones detectees mais plan d'entree invalide (RR ou note insuffisante).",
price)}
    return {"signals": signals, "result": signals[0]}


def analyze_symbol_completely(symbol: str, snapshot: Dict[str, Any]) -> Dict[str, Any]:
    """Resultat principal (meilleur signal) ou motif de rejet."""
    try:
        return find_zone_signals(symbol, snapshot)["result"]
    except Exception as exc:
        logger.exception("Erreur d'analyse sur %s : %s", symbol, exc)
        return build_analysis_error(symbol, "Une erreur inattendue a interrompu l'analyse.",
[str(exc)])


def format_zone_signal_message(signal: Dict[str, Any]) -> str:
    direction = normalize_signal_direction(signal.get("direction"))
    label = "ACHAT (BUY)" if direction == "BUY" else "VENTE (SELL)"
    zone, refined = signal.get("zone", {}), signal.get("refined_zone", {})
    confluences = signal.get("confluences") or ["Zone S/R seule"]
    far = signal.get("entry_mode") == "LIMITE_SUR_ZONE"
    lines = [
        "TRADER AI PRO — SIGNAL ANTICIPE",
        "━━━━━━━━━━━━━━━━━━━━━━",
        f"Indice : {signal.get('symbol_name') or get_symbol_name(str(signal.get('symbol',
''')))}",
        f"Symbole : {signal.get('symbol')}",
        f"Direction : {label}   |   Qualite : {signal.get('grade', '?')}",
        f"Prix observe : {format_price(signal.get('current_price'))}",
    ]
    if far:
        lines.append(f"Distance jusqu'a l'entree : {signal.get('distance_atr')} ATR H4 — PLACER
UN ORDRE LIMITE, attendre le prix")
    lines += [
        "━━━━━━━━━━━━━━━━━━━━━━",
        f"Zone S/R : {format_price(zone.get('lower'))} - {format_price(zone.get('upper'))}",
    ]
    if signal.get("refinements"):
```

```
        lines.append(
            f"Zone affinee ({', '.join(signal['refinements'])}) : "
            f"{format_price(refined.get('lower'))} - {format_price(refined.get('upper'))}"
        )
    tp_note = "niveau technique" if signal.get("tp_source") == "NIVEAU_TECHNIQUE" else "RR
minimum (pas de niveau atteignable)"
    lines += [
        f"Entree indicative : {format_price(signal.get('entry'))} ({'ordre limite' if far else
'prix dans la zone'})",
        f"Stop-loss (SL) : {format_price(signal.get('stop_loss'))}",
        f"Take-profit (TP) : {format_price(signal.get('take_profit'))} ({tp_note})",
        f"Ratio risque/rendement : {format_risk_reward(signal.get('risk_reward'))}",
        "━━━━━━━━━━━━━━━━━━━━━━",
        "Confluences detectees :",
    ]
    lines += [f"• {item}" for item in confluences]
    if signal.get("counter_trend"):
        lines.append("⚠ Contre la tendance D1 : prudence.")
    lines += [
        "━━━━━━━━━━━━━━━━━━━━━━",
        "Le scenario peut etre invalide si le prix casse la zone avant l'entree.",
        "Execution automatique : DESACTIVEE",
        "Mode : ALERTE UNIQUEMENT (compte demo, execution manuelle)",
    ]
    return "\n".join(lines)


def find_signals(symbol: str, market_data: Dict[str, Any]) -> List[Dict[str, Any]]:
    """Adaptateur historique du scanner vers l'analyse complète D1/H4/H2/H1."""
    if not isinstance(market_data, dict):
        return []
    snapshot = {
        "symbol": symbol,
        "current_price": market_data.get("current_price"),
        "timeframes": {
            timeframe: market_data.get(timeframe)
            for timeframe in ("D1", "H4", "H2", "H1")
        },
    }
    try:
        found = find_zone_signals(symbol, snapshot)
        signals = found["signals"]
    except Exception as exc:  # une erreur sur un indice ne bloque pas les autres
        logger.exception("Erreur d'analyse sur %s : %s", symbol, exc)
        return []
    if not signals:
        result = found.get("result", {})
        logger.warning(
            "%s : AUCUN SIGNAL - %s | details=%s | erreurs=%s | note minimale=%s | rayon=%s ATR
| contacts min=%s",
            symbol,
            result.get("reason"),
            result.get("details") or result.get("data"),
            result.get("errors"),
            MIN_SIGNAL_GRADE,
            MAX_ALERT_DISTANCE_ATR,
            MIN_ZONE_TOUCHES,
        )
```

```
    output = []
    for item in signals:
        signal = dict(item)
        signal["rr"] = signal.get("risk_reward")
        signal["automatic_execution"] = False
        output.append(signal)
    return output


def format_message(signal: Dict[str, Any]) -> str:
    """Adaptateur de nom pour le formateur officiel des alertes."""
    if not isinstance(signal, dict):
        raise TypeError("Le signal doit être un dictionnaire.")
    if signal.get("setup_type") == SETUP_TYPE_ZONE:
        return format_zone_signal_message(signal)
    payload = dict(signal)
    payload.setdefault("status", "SIGNAL_CANDIDATE")
    payload.setdefault("risk_reward", payload.get("rr"))
    payload["automatic_execution"] = False
    return format_signal_message(payload)
```